Acknowledgement

I would like to express my sincere gratitude to all those who supported and encouraged me throughout the development of this project. Their guidance and assistance have been invaluable, and I am deeply thankful to each one of them.

I am profoundly grateful to the Department of Computer Science, Aligarh Muslim University, for providing me with the opportunity, academic environment, and necessary facilities to successfully carry out this dissertation work.

It is my privilege to record my heartfelt appreciation and deep sense of gratitude to Dr. Asif Irshad Khan, Department of Computer Science, AMU, Aligarh, my project/dissertation supervisor. His expert guidance, insightful suggestions, continuous encouragement, and unwavering support have been instrumental in shaping this work. His mentorship has greatly enriched my learning experience, and I am sincerely thankful for the time and effort he devoted to supervising this project.

I extend my deepest thanks to my parents and family members, whose constant motivation, patience, and belief in me have always been a source of strength and inspiration. Their unconditional support has played a vital role in helping me achieve my goals.

Above all, I am truly grateful to God for granting me the health, perseverance, and determination to successfully undertake and complete this project.

YASH KUSHWAHA

24CAMSA128

GM8060

Abstract

CoBuilder is a real-time collaborative development platform designed to support project creation, team collaboration, live communication, AI-assisted code generation, and synchronized file-tree editing within a single web-based workspace. The system addresses the common challenge faced by students and development teams who must switch between separate tools for coding, communication, project management, and AI assistance. By combining these functions into one platform, CoBuilder provides a more organized and interactive environment for collaborative software development.

The application is implemented using the MERN stack, with React.js and Vite for the frontend, Node.js and Express.js for the backend, MongoDB for persistent data storage, Socket.IO for real-time communication, JSON Web Token authentication for secure access, and Google Gemini API for generative AI assistance. Authenticated users can register, log in, create project workspaces, invite collaborators, exchange messages, use @ai prompts to generate code or explanations, view generated file structures, edit files, save changes, and synchronize updates with other users in the same project room.

The project demonstrates how real-time web technologies and generative AI can be integrated into a practical collaborative coding environment. The system stores project data, messages, collaborators, and generated file trees so that work can continue across sessions. CoBuilder is suitable for academic demonstrations, student software projects, and small-team collaborative development. Future enhancements may include execution sandboxes, Git integration, role-based permissions, CRDT-based fine-grained editing, and advanced AI-assisted code review.

Keywords: CoBuilder, collaborative development environment, MERN stack, Socket.IO, MongoDB, JWT authentication, generative AI, Gemini API, real-time coding, file-tree synchronization.

Table of Contents

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

1.1 Background and Context

Software engineering has shifted from isolated local development toward distributed, cloud-mediated, and highly collaborative workflows. Modern teams regularly work across institutions, time zones, devices, and heterogeneous development environments. This distribution creates a persistent demand for platforms that can support shared project context, real-time editing, synchronized communication, and rapid feedback. Early groupware research established that collaborative systems differ fundamentally from ordinary multi-user systems because they must support simultaneous actions on shared artifacts while preserving responsiveness and a coherent user experience [1]. A code editor is especially sensitive to this requirement because the edited artifact is not only a document but also an executable, syntactically structured, and semantically meaningful software component.

Cloud-based development environments reduce the burden of local installation and help users access projects through a browser. However, many cloud IDEs and collaboration platforms separate the act of editing code from related development tasks such as requesting explanations, reviewing code, discussing implementation choices, sharing generated files, or debugging errors. In practice, developers often move among a chat application, source control platform, static analysis tool, documentation site, local IDE, and AI assistant. The cognitive cost of switching between tools is substantial, particularly for students and novice developers who are still forming mental models of software architecture.

COBUILDER addresses this problem by integrating real-time collaboration and generative AI assistance into a unified MERN-stack application. The platform enables authenticated users to create project rooms, invite collaborators, exchange messages, request AI-generated code by using an `@ai` command, receive a structured file tree, inspect and edit generated files, persist messages and project state, and synchronize manual file-tree changes with other room members. The system uses a Node.js and Express.js backend, Socket.IO for event-driven real-time communication, MongoDB for persistence, JWT for authentication, Redis for optional token blacklisting during logout, and Google Gemini integration for generative AI services.

The technical significance of COBUILDER lies in its combination of three research areas. First, it draws from collaborative editing and groupware systems, including the challenges of consistency, causality, and user intention [1], [3]. Second, it uses WebSocket-based communication to support low-latency bidirectional events, a design direction motivated by the need for persistent interactive communication in real-time applications [18], [19]. Third, it

incorporates generative AI as an embedded software engineering assistant, a direction supported by recent research on LLM-based code generation, code review, and developer automation [9]-[14].

1.1.1 Conceptual Scope

The conceptual scope of COBUILDER can be represented as an intersection of cloud IDE functionality, real-time groupware, and AI-assisted development. The system is not merely a chat application, not only a code editor, and not solely an AI interface. It is a combined workspace in which communication, project state, file structure, and AI assistance remain within the same collaborative context.

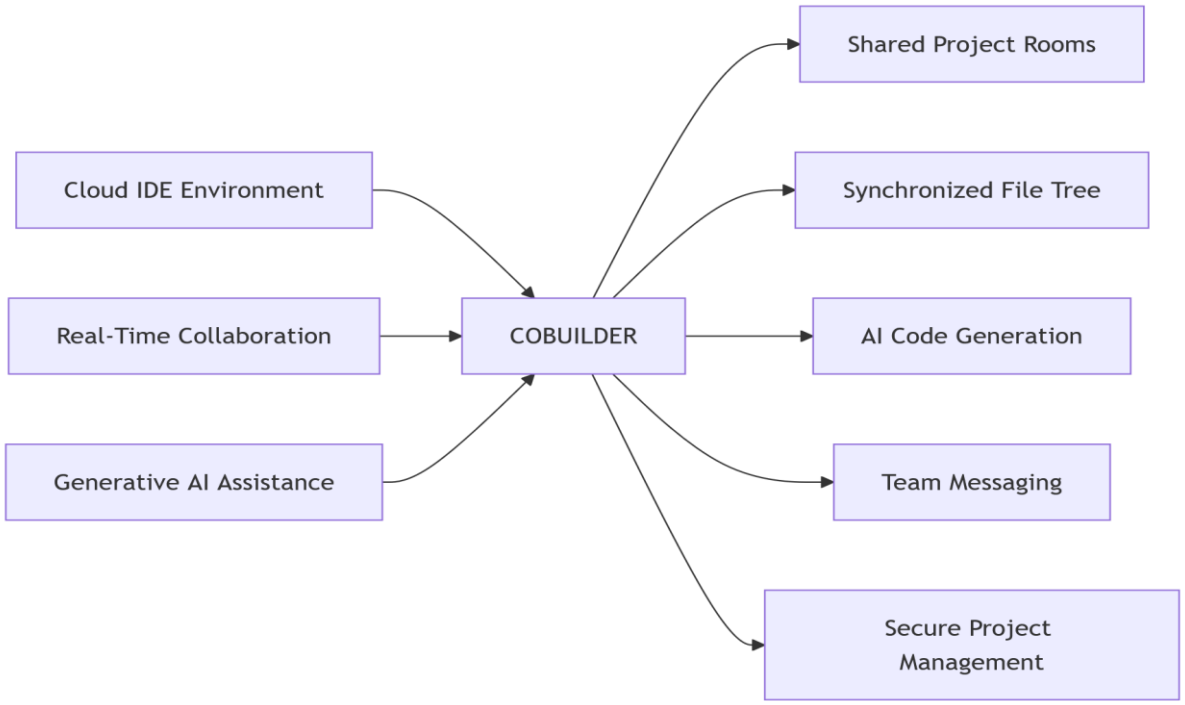

Fig. 1.1. Conceptual scope of COBUILDER.

1.1.2 Project Feature Overview

Table 1.1. Core COBUILDER features.

Feature	Implementation Mechanism	Academic Relevance
Real-time messaging	Socket.IO `project-message` event	Supports synchronous group awareness
Project rooms	Socket room based on MongoDB project identifier	Enforces project-scoped collaboration
AI code generation	Gemini model through backend service	Integrates LLMs into SDLC workflows
File-tree synchronization	`update-file-tree` and `file-tree-updated` events	Maintains shared project state
Authentication	JWT and Express middleware	Protects project access
Persistence	MongoDB project and user collections	Preserves workspace history
Frontend IDE view	React components for file tree and file viewer	Provides browser-based code interaction
Collaborator management	REST endpoint to add project users	Supports team formation
Logout support	Redis token blacklist where available	Improves session control

1.2 Research Problem

The primary research problem addressed in this thesis is the design and implementation of a collaborative cloud development environment that can combine real-time human collaboration with integrated generative AI assistance without fragmenting the developer workflow. Existing solutions frequently solve only part of the problem. Some systems provide shared editing but limited AI support; others offer AI chat or code generation but do not preserve outputs as

synchronized project artifacts; and many educational or lightweight platforms lack robust room membership, authentication, and persistence.

The project does not attempt to solve the full theoretical problem of peer-to-peer text editing under arbitrary network partitions. Instead, it adopts a server-mediated model suitable for a practical final-year MERN project. This choice aligns with the system requirements: authenticated project rooms, database persistence, AI service mediation, and a manageable implementation boundary. The design recognizes the importance of consistency literature but applies it pragmatically through centralized validation and broadcast.

1.3 Significance of Study

The significance of this study is both academic and practical. Academically, COBUILDER provides a concrete system through which established research on groupware, collaborative editing, and AI-assisted software engineering can be studied in an integrated setting. Foundational work on operational transformation demonstrates that collaborative systems must carefully manage concurrent operations to preserve user intention [1], [3]. CRDT research further shows how distributed systems can guarantee convergence through mathematically structured replicated data types [5]. LLM studies demonstrate that code-oriented models can generate, evaluate, and repair code, while also introducing risks related to correctness, sustainability, and overreliance [9]-[14].

Practically, COBUILDER is significant because it reflects the needs of student teams, hackathon groups, remote development teams, and educational laboratories. A beginner-friendly collaborative environment can shorten setup time, centralize discussions, and provide AI assistance without requiring users to install a heavy local development stack. The platform can also be extended with execution sandboxes, version-control integration, CRDT-based editor operations, automated testing, and retrieval-augmented code review.

The broader software engineering significance is that generative AI is most useful when it is integrated into existing engineering contexts rather than used as a detached text generator. COBUILDER therefore treats the AI response as a structured project artifact. When the backend receives an AI response containing a `fileTree`, it broadcasts that structure to users, stores the resulting workspace state, and enables users to inspect or modify generated files.

This design supports the principle that AI-generated code should remain reviewable, editable, and accountable within the project lifecycle.

1.4 Research Questions

This thesis is guided by the following research questions:

RQ1: How can a MERN-stack architecture support real-time collaborative development through secure project rooms and low-latency event synchronization?

RQ2: How can generative AI be integrated into a collaborative development environment so that AI outputs become structured, inspectable, and persistent software artifacts?

RQ3: What architectural trade-offs arise when using a server-mediated Socket.IO synchronization model instead of fully distributed OT or CRDT-based editing?

RQ4: How does the integration of messaging, project management, file-tree visualization, and AI assistance improve the usability of a cloud development environment?

RQ5: What security, scalability, and maintainability considerations are required for a collaborative AI-assisted code platform?

1.5 Objectives

The objectives of the COBUILDER project are as follows:

1. To design a real-time collaborative development environment using React.js, Node.js, Express.js, MongoDB, and Socket.IO.

2. To implement JWT-based authentication and authorization for secure access to project rooms.

3. To provide multi-user project management with collaborator invitation and project-scoped membership.

4. To implement real-time chat and project event broadcasting using WebSocket-based Socket.IO communication.

5. To integrate a generative AI model capable of returning developer assistance, code explanations, and structured file-tree outputs.

6. To persist project messages, collaborators, and AI-generated file trees in MongoDB.

7. To design a responsive frontend interface using React, Vite, Tailwind CSS, and component-based UI architecture.

8. To evaluate COBUILDER through functional validation, comparative analysis, and analytical performance reasoning.

9. To identify future improvements such as CRDT-based fine-grained editing, execution containers, Git integration, and retrieval-augmented project awareness.

1.6 Thesis Statement

This thesis argues that a real-time collaborative development environment becomes more effective when communication, project state, and generative AI assistance are integrated into a single authenticated workspace. By combining Socket.IO-based synchronization, MongoDB-backed project persistence, structured AI file-tree generation, and React-based interactive project views, COBUILDER demonstrates a practical architecture for collaborative software development that is technically feasible, extensible, and aligned with contemporary research in groupware and AI-assisted software engineering.

CHAPTER 2

LITERATURE REVIEW

2.1 Theoretical Framework

The theoretical framework for this thesis combines collaborative editing theory, distributed systems consistency, real-time web communication, and LLM-assisted software engineering. COBUILDER is situated within the category of collaborative development environments, where the shared artifact is a software project rather than a simple text document. The theoretical challenge is therefore broader than broadcasting chat messages; it includes user membership, shared project state, generated artifacts, conflict visibility, and AI-mediated code production.

2.1.1 Groupware and Real-Time Collaboration

Groupware systems are computer-based systems that support multiple users engaged in a common task and provide an interface to a shared environment [1]. In groupware, users must perceive a coherent shared state while retaining the ability to act without excessive delay. Ellis and Gibbs emphasized the need for responsive concurrency control that avoids locking whenever possible [1]. This insight is important for COBUILDER because software collaboration becomes frustrating if every edit requires explicit lock negotiation.

The Jupiter collaboration system further demonstrated the importance of supporting shared documents and shared tools in network-constrained environments [2]. Although Jupiter was developed before modern browser tooling, its focus on latency, bandwidth, shared state, and server-mediated support remains relevant. COBUILDER similarly uses a central server as the coordination point for project rooms, message broadcasting, and AI-mediated file-tree updates.

2.1.2 Consistency in Collaborative Editing

Collaborative editing research identifies three central consistency properties: convergence, causality preservation, and intention preservation [3]. Convergence means that all replicas eventually reach the same state after executing the same set of operations. Causality preservation means operations are executed in an order consistent with their causal dependencies. Intention preservation means the effect of an operation remains consistent with the user's original intention despite concurrent operations [3]. In COBUILDER, chat messages and file-tree replacement events are managed by the server and persisted in MongoDB. This architecture avoids many low-level text-editing conflicts because file updates are saved and

broadcast as explicit events. However, it does not yet implement character-level operational transformation. Therefore, the thesis treats COBUILDER as a pragmatic collaborative development platform and identifies fine-grained editor consistency as a future extension.

2.1.3 Operational Transformation

Operational Transformation (OT) is a technique that transforms concurrent editing operations so that replicas can converge while preserving user intentions [1], [3]. In an OT-based text editor, operations such as insert and delete are transformed against concurrent operations before execution.

OT is powerful but complex because it requires careful operation semantics, concurrency tracking, transformation properties, and edge-case handling. Xue, Orgun, and Zhang explored a multi-versioning approach for preserving intentions in distributed group editors [4]. Such work demonstrates that collaboration correctness can become highly nuanced when multiple users modify the same artifact concurrently.

2.1.4 Conflict-Free Replicated Data Types

CRDTs provide another theoretical approach to collaborative consistency. Shapiro et al. define convergent and commutative replicated data types that allow replicas to be updated independently while guaranteeing eventual convergence under defined assumptions [5]. In text editing, CRDT approaches such as replicated growable arrays and continuous sequence structures assign stable identifiers to inserted elements so concurrent edits can be merged without a central transformation server.

where (\sqcup) is a merge operation that is commutative, associative, and idempotent. These algebraic properties support eventual consistency across replicas. AndrÃƒÆ’Ã‚Â© et al. discuss adaptable granularity of changes for massive-scale collaborative editing, showing that operation granularity affects performance and scalability [6].

COBUILDER currently uses Socket.IO room broadcasting rather than CRDT merging. This is appropriate for its implemented architecture because the server is trusted to serialize events, authorize membership, and persist the latest file-tree state. Nevertheless, CRDT literature provides an important benchmark for future improvement, especially if COBUILDER evolves toward simultaneous character-level code editing.

2.1.5 Real-Time Web Communication

WebSocket communication enables persistent, full-duplex interaction between client and server. In contrast to request-response HTTP polling, WebSocket-based communication allows either side to push events as soon as state changes occur. Socket.IO builds on this model by providing event names, rooms, reconnection handling, and fallbacks, making it practical for collaborative applications. Recent application papers on WebSocket and Socket.IO emphasize their suitability for real-time chat, collaborative interfaces, and low-latency web applications [18], [19].

In COBUILDER, Socket.IO is used for three primary purposes: authenticating a client into a project-specific room, broadcasting chat messages, and synchronizing file-tree updates. The backend validates the project identifier and JWT token during the Socket.IO handshake. This prevents unauthorized clients from joining arbitrary rooms.

2.1.6 Generative AI and Software Engineering

Large language models have introduced new possibilities for software engineering. Chen et al. evaluated large language models trained on code and demonstrated the ability of such models to synthesize programs from natural language prompts [9]. More recent surveys describe LLM-based agents as systems that can decompose tasks, generate code, debug, and participate in broader software lifecycle workflows [10]. Research on sustainable code generation further cautions that generated code should be assessed for efficiency, maintainability, and environmental impact [11].

Code review automation is another important application area. Context-aware code review work shows that LLMs can be combined with retrieval mechanisms to generate review comments grounded in historical examples [12]. Studies of LLMs as code reviewers report promising results in defect detection, style analysis, and security review, while still requiring human oversight [13]. COBUILDER applies these ideas by embedding AI directly in the workspace and requiring generated files to be inspected and edited by users.

2.2 Current State of Knowledge

The current state of knowledge can be grouped into four areas: collaborative editing algorithms, real-time development environments, cloud collaboration security, and AI-assisted coding.

2.2.1 Collaborative Editing Algorithms

The classical literature establishes the conceptual foundation. Ellis and Gibbs identified responsive groupware concurrency control as an alternative to strict locking [1]. Sun et al. formalized convergence, causality preservation, and intention preservation [3]. Xue et al. proposed multi-versioning to preserve concurrent user intentions [4]. Shapiro et al. generalized replicated data types through CRDT theory [5]. AndrÃƒÆ'Ã‚Â© et al. examined how change granularity impacts collaborative editing scalability [6].

These works reveal that collaborative editing is not only a networking problem. It is a semantic state-management problem in which operations must preserve meaning across users and replicas. COBUILDER currently handles collaboration at the level of messages and file-tree state, which is less complex than character-level editing but still requires project-scoped authorization, event ordering, persistence, and UI synchronization.

2.2.2 Real-Time Collaborative Development Tools

Recent papers demonstrate continued interest in collaborative code editors and real-time web IDEs. A comprehensive study on real-time web IDE collaborative code editors highlights the need for concurrency management, conflict resolution, and user awareness features [16]. CodeSync, a real-time collaborative code editor, uses WebSocket communication and CRDTs to enable synchronous browser-based editing [17]. LiveCollab and unified multi-modal collaborative development environments emphasize the value of combining code editing with file synchronization, communication, and shared project tooling [20], [21].

COBUILDER shares this direction but adds an AI-integrated workflow. Instead of treating collaboration as only shared editing, COBUILDER treats AI output as part of the collaborative project stream. When a user prompts the AI, the response can produce both text and a structured file tree. This file tree is then visible to collaborators and becomes part of the project state.

2.2.3 Cloud Trust and Security

Cloud collaboration creates trust concerns because project data may reside on server infrastructure. Feldman et al. presented SPORC as a framework for group collaboration using untrusted cloud resources, demonstrating that collaborative applications can be designed to reduce trust in cloud providers through cryptographic protections [8]. COBUILDER does not implement SPORC-style cryptographic verification, but its security model includes JWT authentication, project membership checks, CORS restrictions, MongoDB persistence, password hashing, and Redis-backed logout blacklisting where available.

The security requirements for COBUILDER are pragmatic. The platform must ensure that only authenticated users access project APIs, only project members can add collaborators, and only authorized socket clients can join a project room. These controls are necessary before more advanced protections, such as end-to-end encryption or cryptographic audit logs, can be added.

2.2.4 AI-Assisted Coding and Review

LLM-based coding tools are increasingly capable of code generation, debugging, documentation, and review. Code-generation agents can extend beyond snippet creation toward SDLC-level support, including task planning and tool integration [10]. Sustainable code-generation research warns that generated code should be evaluated for execution time, CPU usage, memory consumption, and energy impact [11]. Retrieval-augmented code review research indicates that contextual grounding can improve review relevance and reduce hallucination [12].

COBUILDER's AI integration is intentionally structured. The AI service requests JSON output with fields such as `text`, `fileTree`, `buildCommand`, and `startCommand`. This design reduces ambiguity and allows the frontend to render generated files rather than showing only free-form text. Structured generation also supports future validation, static analysis, and test execution.

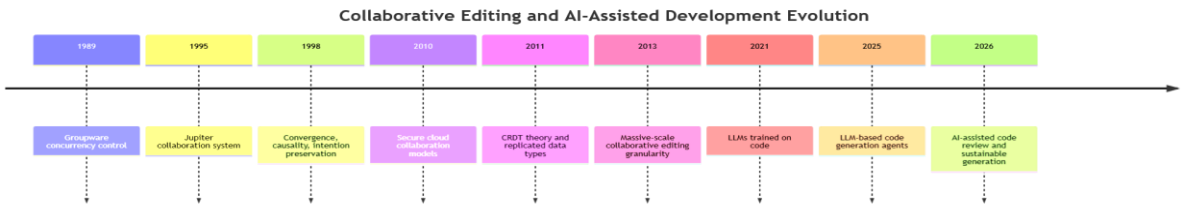

Fig. 2.1. Evolution of collaborative editing technologies.

Table 2.1. Comparative summary of selected literature.

Ref.	Work	Main Contribution	Relevance to COBUILDER
[1]	Ellis and Gibbs	Concurrency control for groupware	Establishes responsive shared editing principles
[2]	Jupiter	Shared tools under network constraints	Supports server-mediated collaboration rationale
[3]	Sun et al.	Consistency model for cooperative editing	Defines convergence, causality, and intention preservation
[4]	Xue et al.	Multi-version intention preservation	Highlights complex concurrent-edit semantics
[5]	Shapiro et al.	CRDT theory	Provides future direction for fine-grained editing
[6]	AndrÃƒÆ'Ã‚Â© et al.	Change granularity for massive collaboration	Supports scalability discussion
[8]	SPORC	Cloud collaboration with untrusted servers	Motivates security and trust analysis
[9]	Chen et al.	LLMs trained on code	Supports AI code generation
[10]	Dong et al.	LLM-based code generation agents	Supports agentic SDLC perspective
[12]	Icoz and Biricik	RAG-based code review automation	Supports AI review and debugging features

2.3 Research Gaps

The literature reveals several gaps that motivate COBUILDER.

First, many collaborative editing systems focus on algorithmic consistency but do not address integrated AI assistance. OT and CRDT research is mathematically rich, yet it usually treats the shared artifact as human-edited text. In modern development environments, AI-generated

code may enter the collaborative state as a large structured artifact. This creates new questions: how should generated files be represented, reviewed, persisted, and synchronized?

Second, many AI coding tools are individual-user assistants. They can produce code, but their outputs are often copied manually into a project. In team settings, this creates ambiguity about authorship, review responsibility, and shared visibility. COBUILDER addresses this by broadcasting AI-generated artifacts inside the project room.

Third, lightweight collaborative development platforms often lack secure project membership. A collaborative room is useful only if access is controlled. COBUILDER uses JWT authentication and validates project identifiers during Socket.IO connection setup.

Fourth, educational project environments often prioritize demonstration over maintainability. COBUILDER is designed with separated controllers, services, models, routes, and frontend components. This modularity improves traceability and supports academic evaluation.

Fifth, current AI-assisted code review research often focuses on pull requests or offline datasets [12], [13]. COBUILDER explores how AI assistance can be used earlier, during collaborative design and generation, before code reaches a repository review stage.

2.4 Key Concepts and Definitions

Collaborative Development Environment: A software environment that enables multiple developers to work on shared project artifacts, communicate, and coordinate development tasks.

Cloud IDE: A browser-accessible development environment where code editing and project management occur through web interfaces rather than local installations.

Socket.IO Room: A server-side grouping mechanism that allows events to be emitted only to clients associated with a specific room.

Generative AI: Artificial intelligence capable of producing new text, code, explanations, or structured artifacts from prompts.

File Tree: A hierarchical representation of project files and directories. In COBUILDER, the file tree is represented as a JSON object in which files contain content under a `file.contents` field.

Project State: The persistent and active state of a COBUILDER project, including collaborators, messages, file tree, build commands, and start commands.

Convergence: The property that multiple replicas eventually reach the same state after receiving the same updates [3], [5].

Causality Preservation: The property that operations are applied in an order consistent with their causal relationships [3].

Intention Preservation: The property that the effect of a user operation remains consistent with the user's intended action despite concurrent operations [3], [4].

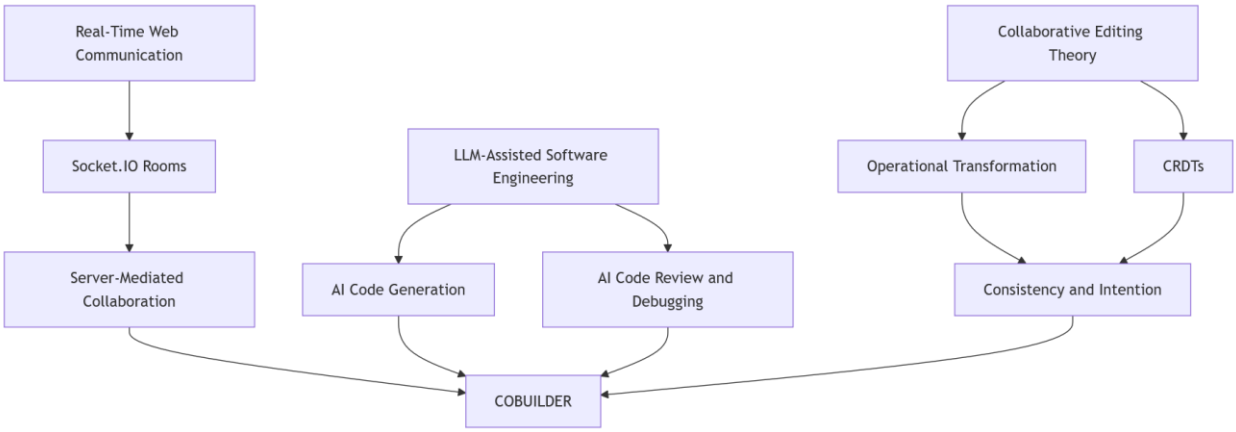

Fig. 2.2. Relationship among OT, CRDT, WebSocket synchronization, and LLM assistance.

Table 2.2. OT and CRDT comparison.

Criterion	**Operational Transformation**	**CRDT**
Primary idea	Transform concurrent operations	Design operations/states to merge safely
Coordination model	Often server-based or causality-aware	Can support decentralized replication

Criterion	Operational Transformation	CRDT
Correctness focus	Convergence, causality, intention	Eventual convergence by algebraic properties
Implementation complexity	High for rich operations	High for identifiers, metadata, garbage collection
Suitability for COBUILDER v1	Future fine-grained editor extension	Future distributed editing extension
Current COBUILDER approach	Server serializes project events	File-tree events are broadcast and persisted

Table 2.3. LLM-assisted software engineering literature mapping.

Research Area	Representative Works	COBUILDER Design Implication
Code generation	Chen et al. [9], Dong et al. [10]	Natural language prompts can create project code
Sustainable generation	Masthan Ali et al. [11]	Generated code must be reviewed for efficiency
Code review automation	Icoz and Biricik [12], Popoola [13]	AI review should be contextual and human-supervised
LLM orchestration	Syed [14]	Multi-stage AI workflows can support data/code engineering
AI in education	Wegerif and Casebourne [15]	AI should support learning dialogue, not replace understanding

CHAPTER 3
METHODOLOGY

3.1 Research Design

The methodology of this thesis follows a design science and applied software engineering approach. The objective is not only to analyze existing literature but to construct and evaluate a working artifact. The artifact is the COBUILDER platform. The research design includes requirement analysis, architecture design, implementation, integration, and analytical evaluation.

Design science is appropriate because the project addresses a practical problem: the need for an integrated collaborative development environment with AI support. The project is evaluated through functional validation, comparative analysis, and technical reasoning. Since the application is a final-year major project, the evaluation emphasizes engineering completeness, architectural justification, and traceability rather than a large-scale controlled user study.

The development process followed an iterative model:

1. Requirement identification from the project objective and literature.
2. Selection of MERN stack and Socket.IO as the implementation foundation.
3. Backend design for users, projects, AI service, REST APIs, and WebSocket events.
4. Frontend design for authentication, dashboard, project room, messaging, file tree, and file viewer.
5. Integration of generative AI through a backend service.
6. Persistence of project messages and file trees in MongoDB.
7. Functional testing and analytical evaluation.

3.1.1 System Requirement Specification

The functional requirements are:

FR1: The system shall allow users to register and log in using email and password.

FR2: The system shall authenticate users using JWT.

FR3: The system shall allow authenticated users to create projects.

FR4: The system shall allow project owners or members to add collaborators.

FR5: The system shall allow users to open a project workspace.

FR6: The system shall establish a Socket.IO connection for a valid project and token.

FR7: The system shall broadcast project messages to all connected users in the room.

FR8: The system shall process AI prompts when a message contains `@ai`.

FR9: The system shall broadcast AI responses and file-tree outputs.

FR10: The system shall persist messages and file trees in MongoDB.

FR11: The system shall support manual file editing and file-tree synchronization.

The non-functional requirements are:

NFR1: The system should provide low-latency event propagation under ordinary network conditions.

NFR2: The system should isolate project rooms by authorization and project identifier

NFR3: The frontend should be responsive and usable on modern browsers.

NFR4: The backend should follow modular separation of concerns.

NFR5: The AI response format should be structured enough to support file rendering.

NFR6: The system should support deployment configuration through environment variables.

3.2 Data Collection Methods

Data for the thesis was collected from three sources:

1. **Project source code:** The actual COBUILDER implementation was analyzed, including backend routes, controllers, services, models, Socket.IO server logic, AI service configuration, and frontend project-room components.

2. **Research papers:** Literature was reviewed from the supplied research-paper folder. The papers cover groupware concurrency control, operational transformation, CRDTs, cloud collaboration, WebSocket communication, collaborative code editors, and LLM-assisted software engineering.

3. **Analytical measurements and assumptions:** Since the thesis is based on a project prototype, performance evaluation uses analytical metrics such as event path length, expected synchronization delay, payload structure, and functional validation scenarios.

No personal user data, live production logs, or external human-subject data were collected. The evaluation therefore remains a technical artifact evaluation rather than a human-subject study.

3.3 Sampling Strategy

The literature sample was selected using relevance to the project. Papers were grouped into four categories:

1. **Foundational collaboration theory:** Groupware, OT, consistency, and intention preservation [1]-[4].

2. **Distributed consistency models:** CRDTs and collaborative editing granularity [5], [6].

3. **Collaborative development and real-time web systems:** Cloud collaboration, collaborative code editors, WebSocket, Socket.IO, and real-time web IDEs [7], [8], [16]-[21].

4. **Generative AI for software engineering:** LLM code generation, LLM agents, sustainable code generation, automated code review, and AI pedagogy [9]-[15].

The implementation sample consists of the primary modules of COBUILDER. These modules include user authentication, project management, Socket.IO collaboration, AI service integration, frontend workspace, file tree viewer, and file editing workflow.

3.4 Analytical Approach

The analytical approach uses architectural analysis, functional validation, comparative analysis, and metric-based reasoning.

3.4.1 Architectural Analysis

The architecture is analyzed by identifying components, data flows, trust boundaries, API contracts, and real-time event paths. A component is considered well-designed if it has a clear responsibility and interacts through explicit interfaces.

3.4.2 Functional Validation

Functional validation checks whether the implemented system satisfies each functional requirement. Each feature is mapped to the responsible module and expected behavior.

3.4.3 Comparative Analysis

COBUILDER is compared against categories of existing systems, including ordinary local IDEs, chat-only collaboration systems, generic cloud IDEs, and AI-only coding assistants. The goal is to identify how integration changes the workflow.

3.4.4 Performance Reasoning

The expected synchronization latency is modeled as:

$$L_{sync} = L_{client\_emit} + L_{network\_up} + L_{server\_process} + L_{network\_down} + L_{client\_render}$$

where:

L_{sync} = total synchronization delay

L_{client_emit} = time taken by the client to emit a socket event

$L_{network\_up}$ = time taken for the event to travel from client to server

$L_{server\_process}$ = time taken by the server to validate and process the event

$L_{network\_down}$ = time taken for the event to travel from server to other clients

L_{client_render} = time taken by the receiving client to update the interface

For AI-assisted messages, the total delay also includes prompt handling, model inference, JSON parsing, and database writing:

$$L_{ai} = L_{sync} + L_{prompt\_processing} + L_{LLM\_inference} + L_{json\_parsing} + L_{db\_write}$$

where:

L_{ai} = total delay for an AI-assisted message

$L_{prompt\_processing}$ = time taken to prepare the user prompt for the AI service

$L_{LLM\_inference}$ = time taken by the language model to generate a response

$L_{json\_parsing}$ = time taken to parse the AI response into usable project data

L_{db_write} = time taken to store the AI response, message, or file tree in the database

This explains why ordinary chat events should be faster than AI-generated file-tree events.

3.5 Ethical Considerations

AI-assisted development introduces ethical responsibilities. Users may rely on generated code without understanding its limitations. Therefore, COBUILDER should be understood as an assistance platform rather than an authority. AI-generated code may contain defects, security vulnerabilities, license ambiguity, or inefficiencies. Human review remains essential [11]-[13].

Data privacy is also relevant. Project messages and file trees are persisted in MongoDB. In a production setting, project data should be protected using secure database credentials, encrypted transport, access logging, backup policies, and possibly encryption at rest. Secrets must not be hard-coded into source files. API keys for generative AI must be stored in environment variables.

The platform should also prevent abuse of AI endpoints through rate limiting, prompt validation, authorization checks, and usage monitoring. Since students and developers may paste sensitive code into AI prompts, future versions should include warnings and controls for confidential data.

3.6 System Architecture

The COBUILDER architecture consists of a React frontend, an Express backend, a MongoDB database, optional Redis service, Socket.IO real-time channel, and external Gemini AI service.

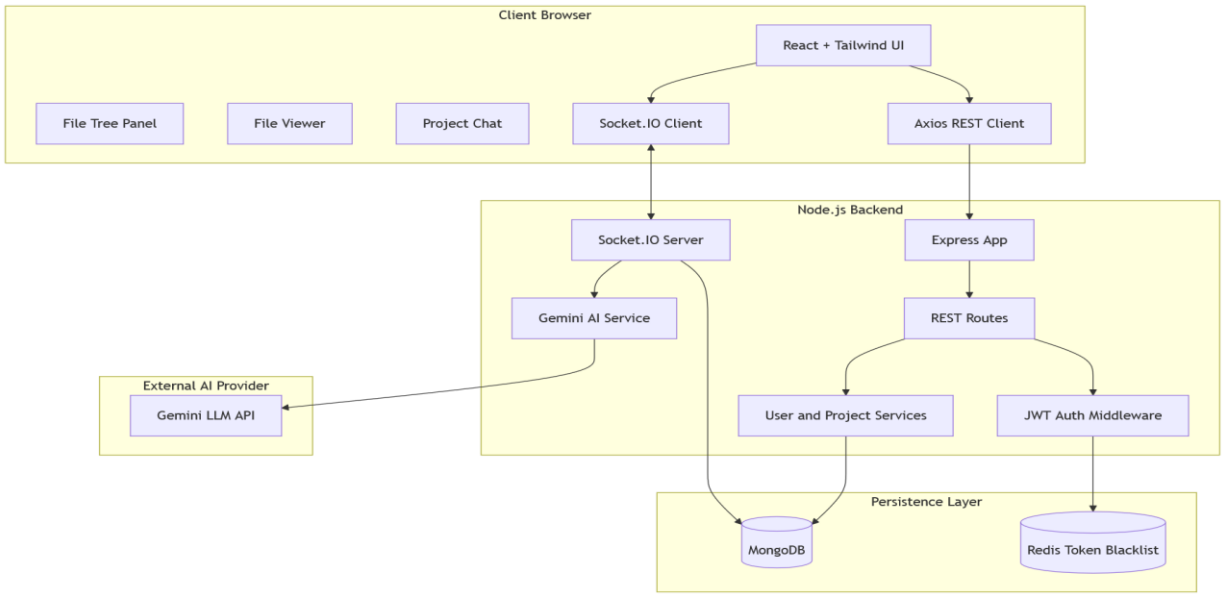

Fig. 3.1. Overall COBUILDER system architecture.

3.6.1 Deployment Architecture

The backend can be deployed as a Node.js web service, while the frontend can be deployed as a static Vite build. Environment variables configure backend URL, MongoDB URI, JWT secret, Google AI API key, Redis credentials, and CORS origins.

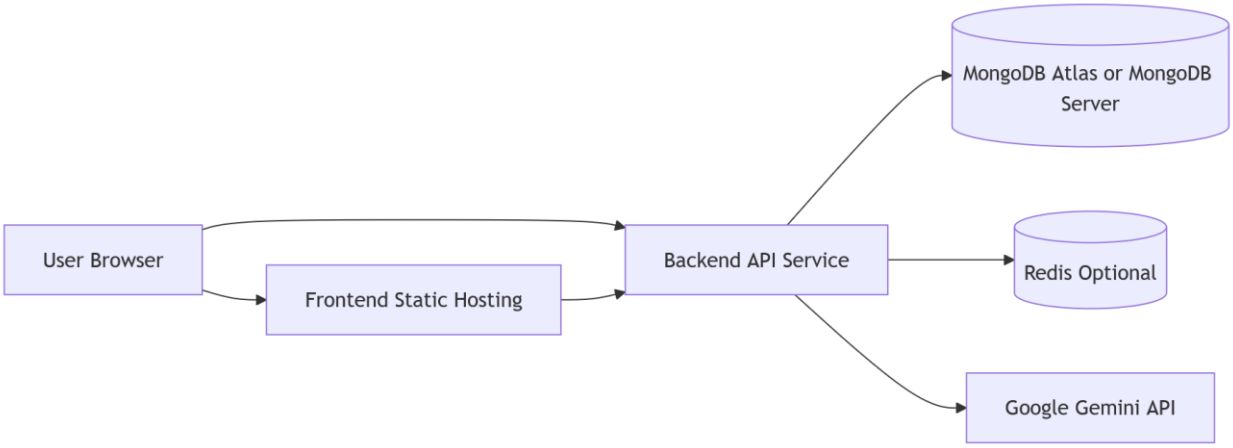

Fig. 3.2. Deployment-level architecture.

3.6.2 Technology Stack

Table 3.1. Technology stack and design rationale.

Layer	**Technology**	**Rationale**
Frontend	React.js	Component-based UI and state management
Styling	Tailwind CSS	Rapid responsive design with utility classes
Build Tool	Vite	Fast development and optimized builds
Backend	Node.js	JavaScript runtime aligned with MERN stack
API Framework	Express.js	Lightweight REST API development
Real-time Layer	Socket.IO	Event-based WebSocket communication and rooms

Layer	Technology	Rationale
Database	MongoDB	Flexible document model for users, projects, file trees
ODM	Mongoose	Schema modeling and validation
Authentication	JWT	Stateless client-server authentication
Password Security	bcrypt	Secure password hashing
Token Blacklist	Redis	Optional logout invalidation support
AI Integration	Gemini API	Generative AI for developer assistance

3.7 UML and Data Modeling

3.7.1 Use Case Diagram

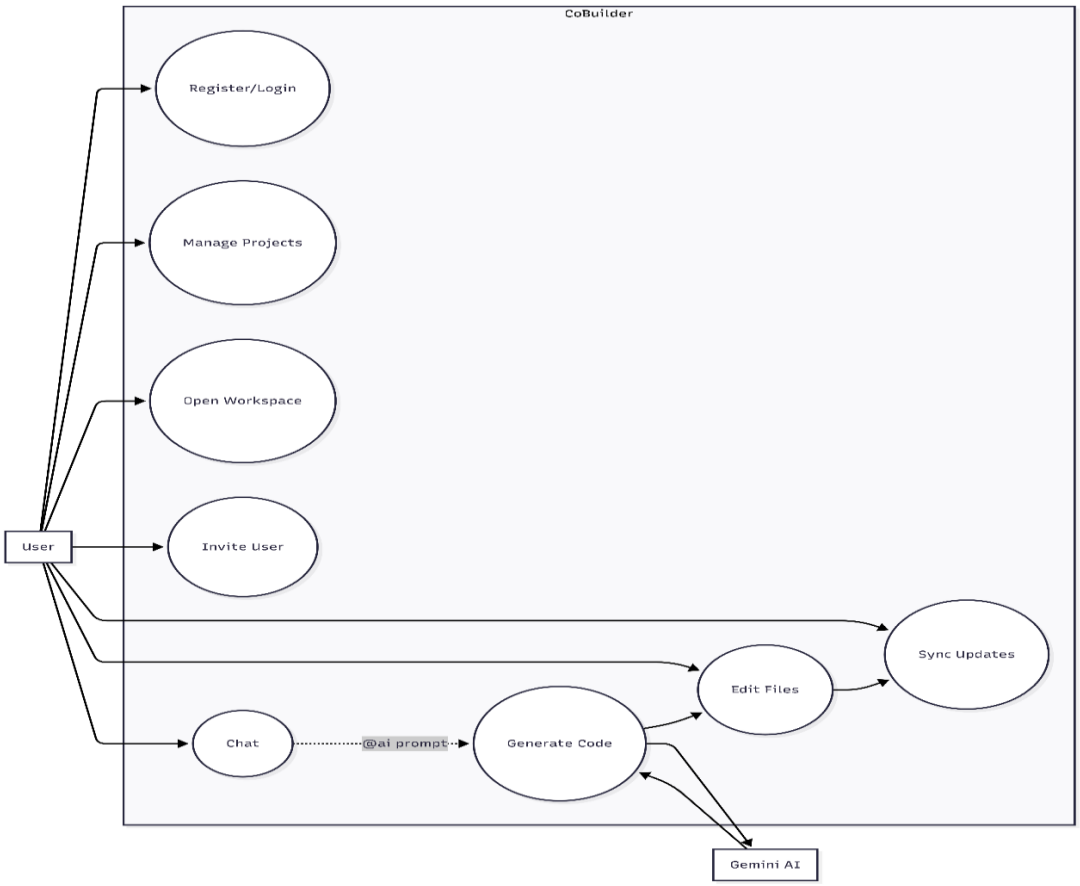

Fig. 3.3. Use case diagram for COBUILDER

3.7.2 Entity Relationship Diagram

The primary entities are User and Project. A project contains multiple users, messages, and an optional file tree. The file tree is modeled as a flexible object because generated project structures can vary across programming languages and frameworks.

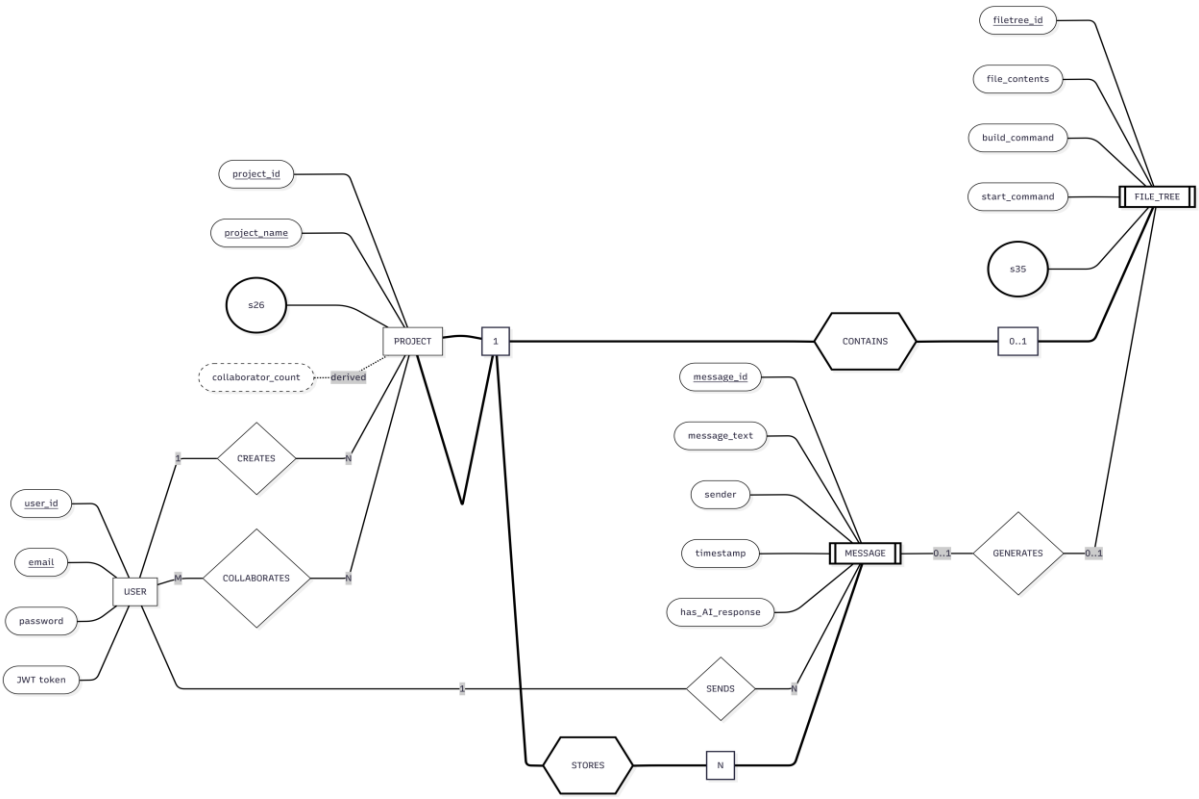

Fig. 3.4. ER diagram for COBUILDER

3.7.3 Data Flow Diagram

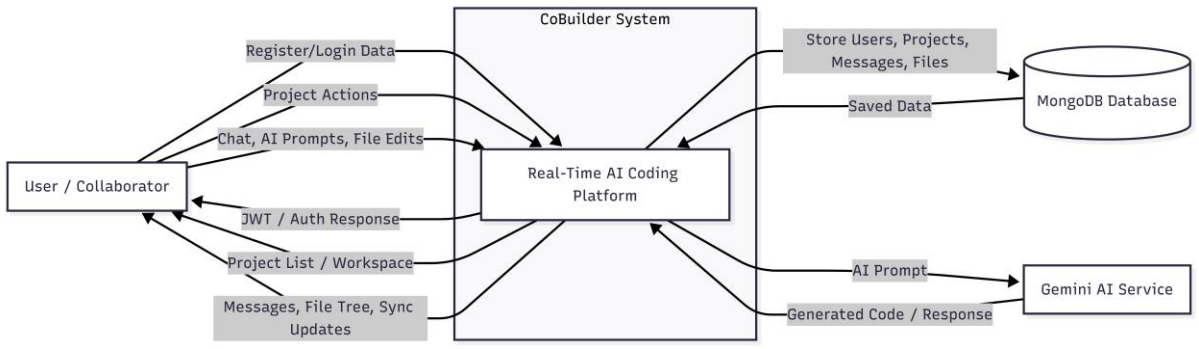

Fig. 3.5. Level-0 data flow diagram.

3.8 Backend Implementation

The backend follows a modular Express structure. The `app.js` file initializes Express middleware, CORS, JSON parsing, URL encoding, cookies, logging, and route registration. The `server.js` file creates the HTTP server, attaches Socket.IO, validates socket clients, and handles real-time project events.

3.8.1 Authentication

Users register with email and password. Passwords are hashed using bcrypt before storage. During login, the backend verifies credentials and returns a JWT. Protected routes use authentication middleware that reads the token from cookies or the Authorization header. If Redis is connected, the middleware checks whether the token is blacklisted due to logout.

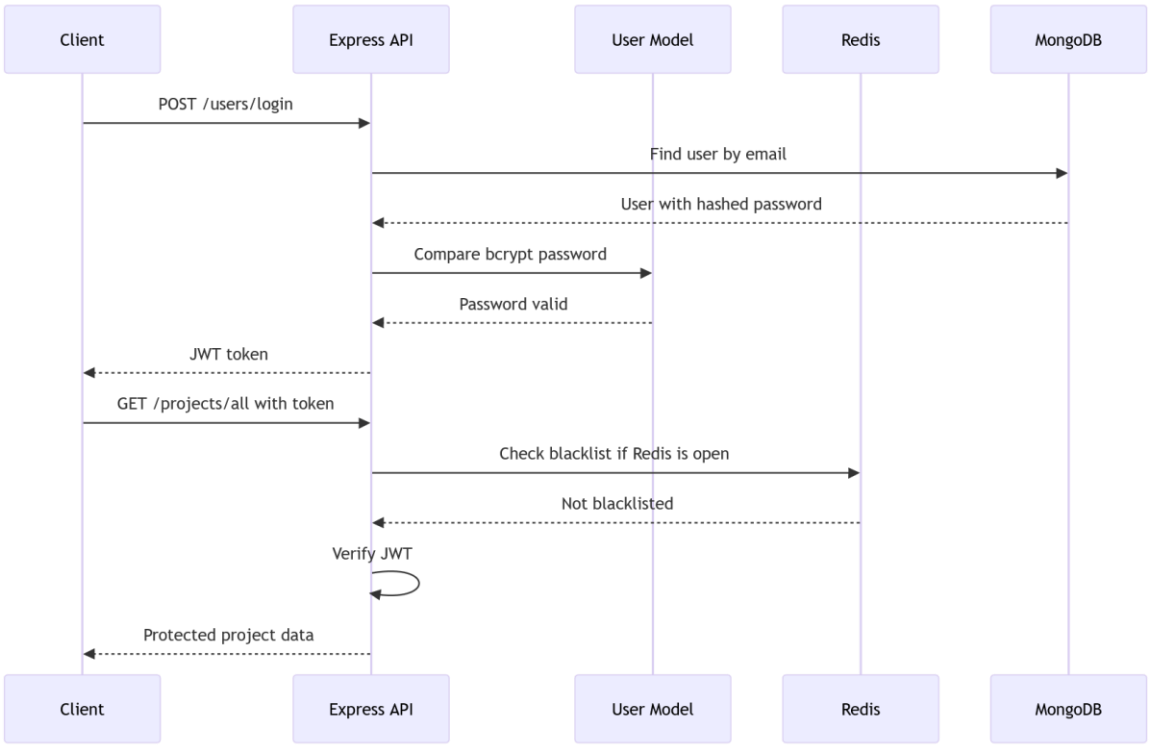

Fig. 3.6. Authentication and authorization sequence.

3.8.2 Project Management

The project model stores the project name, collaborator user identifiers, file tree, and messages. The project service supports project creation, project listing for a user, adding users to a project, and retrieving project details. The `addUserToProject` service validates the project ID, user IDs, and membership of the requesting user before updating collaborators.

3.8.3 Real-Time Collaboration

Socket.IO middleware validates the project ID and JWT before allowing the socket connection to proceed. After connection, the server stores the room ID as the project ID and joins the socket to that room. This design ensures that messages are emitted only to collaborators connected to the same project.

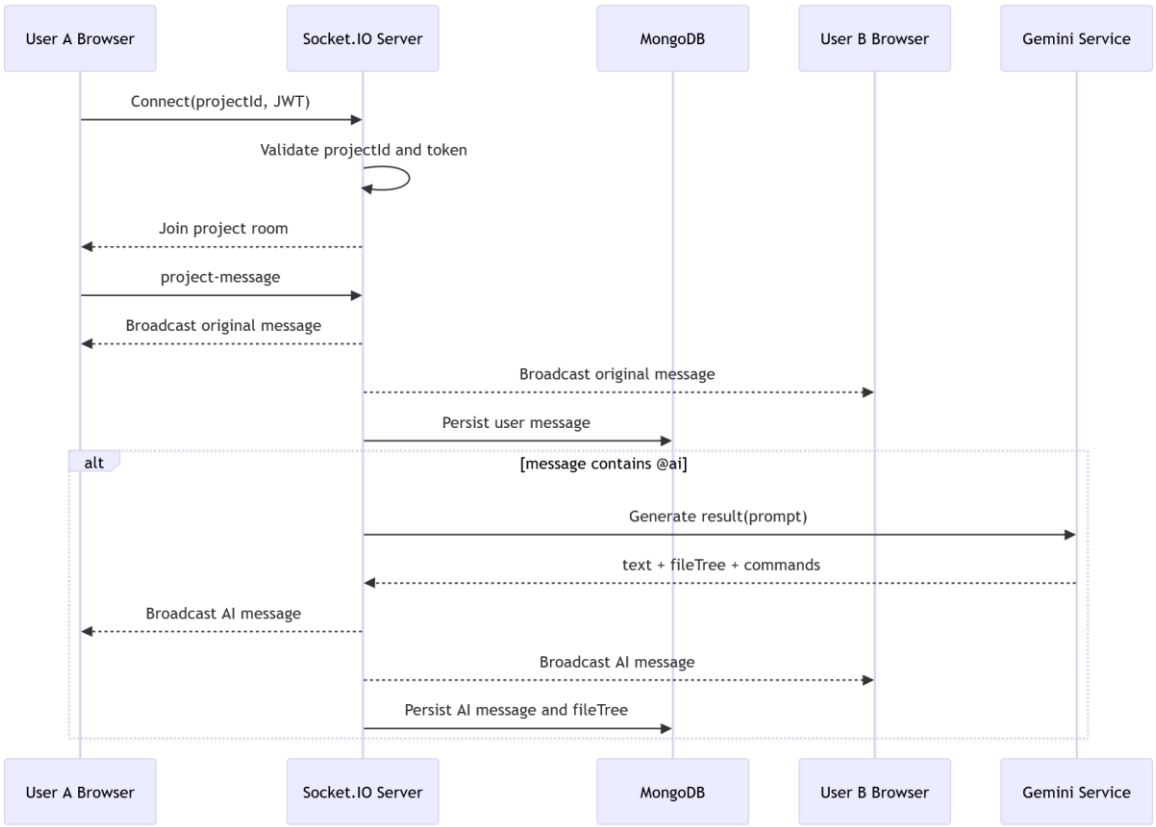

Fig. 3.7. Socket.IO message flow.

3.8.4 AI Service

The AI service uses Google Generative AI and configures the model for JSON output. The system instruction requests modular MERN-oriented responses and structured file-tree generation when users ask to create or build code. The service attempts to parse JSON, extracts `text`, `fileTree`, `buildCommand`, and `startCommand`, and includes fallback parsing for malformed responses.

This structured response is important because natural language alone cannot update the project workspace. The frontend needs a file-tree object to render files, open the first generated file, and allow user edits.

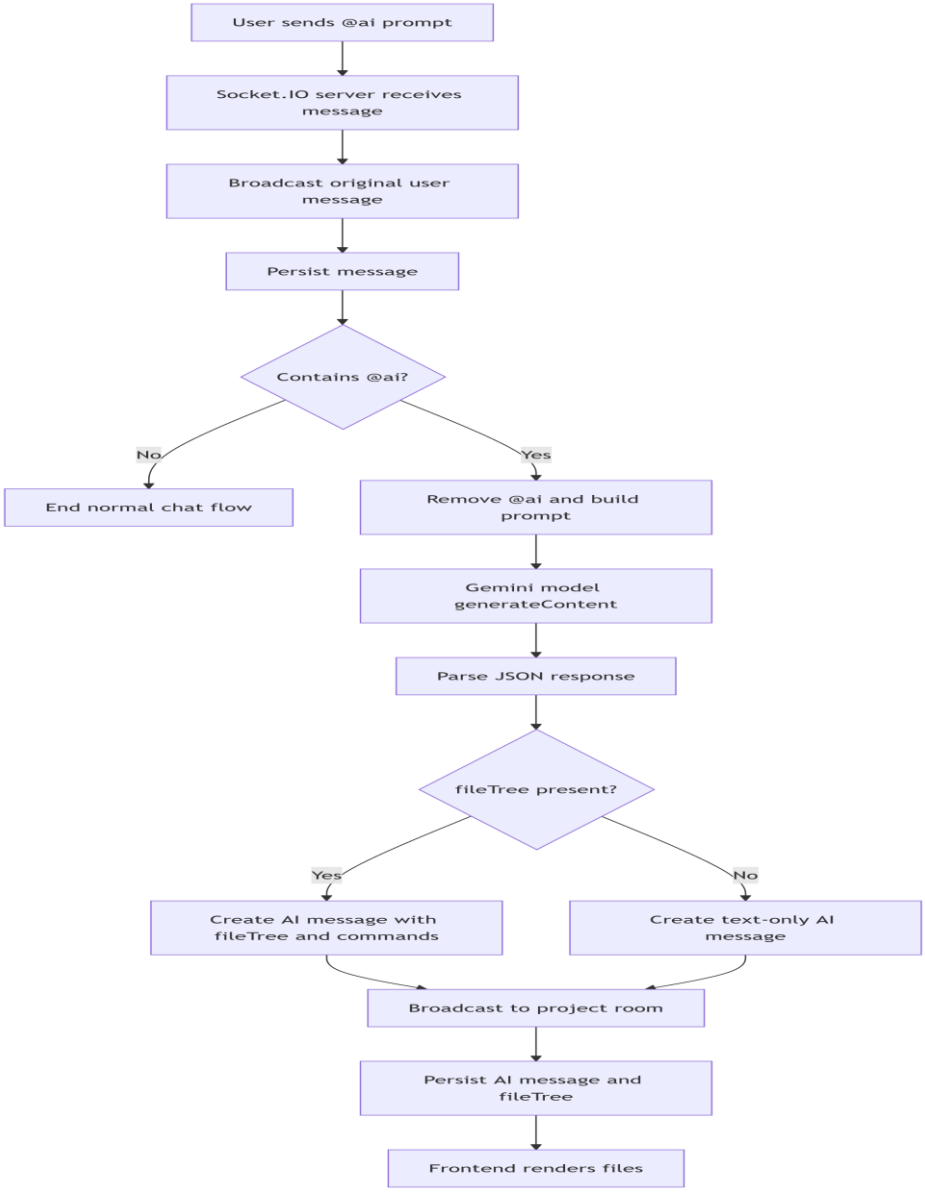

Fig. 3.8. AI-assisted code generation workflow.

3.8.5 API Specification

Table 3.2. REST API specification.

Method	Endpoint	Purpose	Authentication
POST	`/users/register`	Register a new user	No
POST	`/users/login`	Login and receive JWT	No
GET	`/users/profile`	Retrieve authenticated profile	Yes
GET	`/users/logout`	Logout and blacklist token if Redis is available	Yes
GET	`/users/all`	Get all users except current user	Yes
POST	`/projects/create`	Create a project	Yes
GET	`/projects/all`	Get projects for current user	Yes
PUT	`/projects/add-users`	Add collaborators	Yes
GET	`/projects/get-project/:projectId`	Get project detail	Yes
GET	`/ai/get-result/:prompt`	Generate AI response	No in route, should be protected in future

Table 3.3. Socket event specification.

Event	Direction	Payload	Behavior
`connection`	Client to server	JWT and projectId	Validate and join project room
`project-message`	Client to server	Message object	Broadcast and persist message
`project-message`	Server to clients	User or AI message	Render in project chat
`update-file-tree`	Client to server	File tree object	Save tree and broadcast to peers
`file-tree-updated`	Server to clients	File tree object	Update peer workspace
`disconnect`	Client to server	Socket close	Leave project room

Table 3.4. Database schema summary.

Collection	Field	Type	Purpose
users	email	String	Unique user identifier
users	password	String	bcrypt password hash
projects	name	String	Project name
projects	users	ObjectId[]	Collaborator references
projects	fileTree	Object	Current generated/edited project file tree
projects	messages	Array	Project chat and AI message history

3.8.6 Security Controls

Table 3.5. Security controls.

Threat	Control in COBUILDER	Future Enhancement
Unauthorized REST access	JWT middleware	Role-based access control
Unauthorized socket room access	JWT and projectId validation in Socket.IO middleware	Membership check before join
Password exposure	bcrypt hashing	Password complexity and breach checks
Token reuse after logout	Redis blacklist if connected	Refresh-token rotation
Cross-origin abuse	Configurable CORS origins	Strict production allowlist
AI API key exposure	Environment variable	Secret manager integration
Prompt misuse	Backend-mediated AI service	Rate limiting and moderation
Data leakage	Project-scoped API queries	Encryption at rest and audit logs

3.9 Frontend Implementation

The frontend is implemented in React with Vite and Tailwind CSS. It includes screens for registration, login, dashboard, home, and project workspace. The project workspace is the core interface. It loads project details using Axios, establishes a Socket.IO connection, renders messages, shows collaborators, receives AI file trees, displays a file tree panel, opens files in tabs, allows file editing, and emits file-tree updates.

3.9.1 Project Room UI

The project room is divided into a left-side collaboration panel and a right-side code workspace. The chat panel contains the message stream and input field. The code workspace contains the file tree and file viewer when a file tree exists. If no file tree exists, the UI displays an empty workspace state and encourages the user to prompt AI.

3.9.2 File Tree and Editing Model

When an AI message contains a file tree, the frontend updates `currentFileTree`, opens the first available file, stores build and start commands, and displays a success notification. When a user edits and saves a file, the frontend updates the nested file tree object and emits `update-file-tree` to the backend. The backend persists the new tree and broadcasts `file-tree-updated` to other clients in the room.

3.9.3 Sequence for Collaborative Project Work

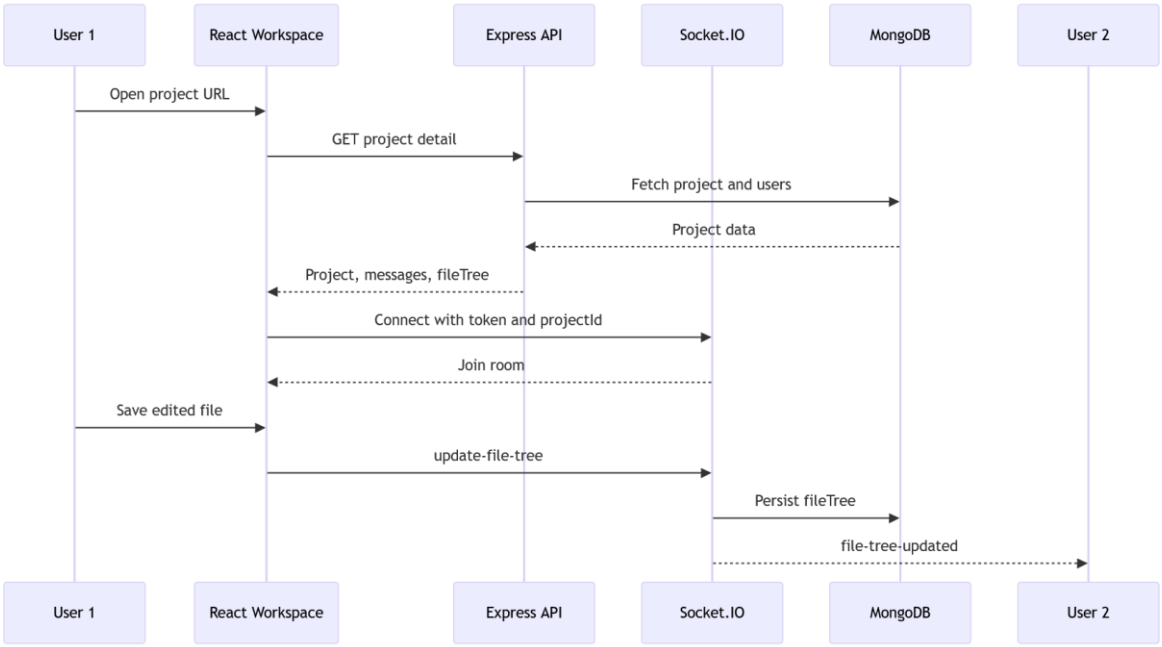

Fig. 3.9. Project-room collaboration sequence.

3.9.4 Activity Diagram

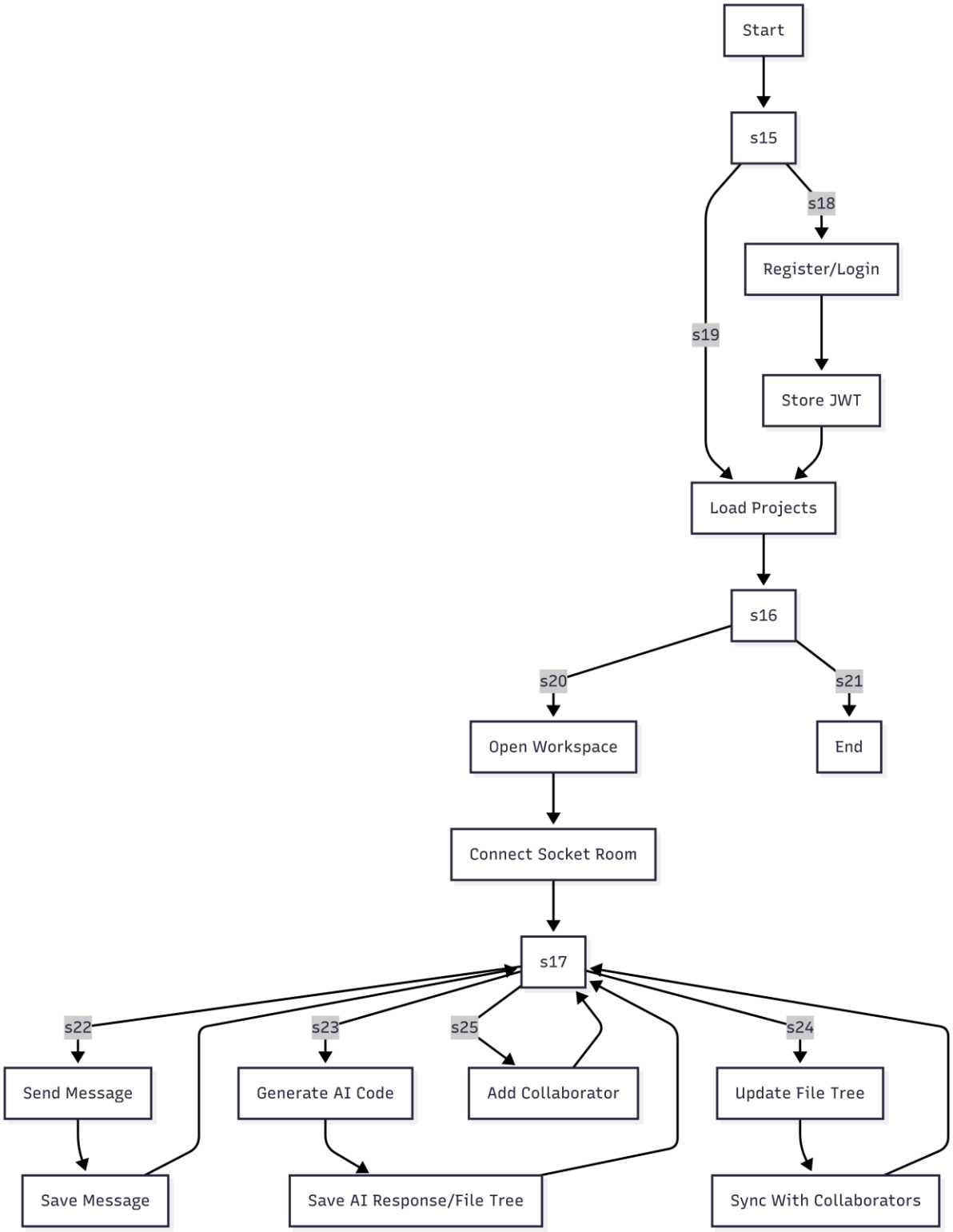

Fig. 3.10. Activity diagram for collaborative code generation.

3.10 Algorithms

3.10.1 Algorithm 1: Socket Authentication and Room Join

Algorithm 1. Socket authentication and project room assignment.

Input: (\text{token}), (\text{projectId})

Output: Accepted socket connection or authentication error

1. Receive socket handshake.
2. Extract JWT token from `handshake.auth.token` or Authorization header.
3. Extract `projectId` from query parameters.
4. Validate that `projectId) is a valid MongoDB ObjectId.
5. Query MongoDB for the project.
6. If project does not exist, reject connection.
7. If token is missing, reject connection.
8. Verify JWT using `JWT_SECRET`.
9. Attach decoded user and project to socket object.
10. Call `next()` and allow connection.
11. On connection, set room ID to project ID and join socket to room.

3.10.2 Algorithm 2: AI Message Handling

Algorithm 2. AI-assisted message processing.

Input: Message (m) from client

Output: Broadcast message and optional AI response

1. Receive `project-message` event.
2. Broadcast original message to all clients in project room.
3. Persist original message in MongoDB.
4. Read `m.text).

5. If `m.text` is not a valid string, stop.

6. If `m.text` does not contain `@ai`, stop.

7. Remove `@ai` token and trim remaining prompt.

8. Send prompt to Gemini AI service.

9. Parse response as structured JSON.

10. Extract `text`, `fileTree`, `buildCommand`, and `startCommand`.

11. Construct AI message object.

12. Broadcast AI message to project room.

13. Persist AI message and update project file tree if present.

3.10.3 Algorithm 3: File Tree Update

Algorithm 3. File-tree synchronization.

Input: Updated file tree (T')

Output: Persisted project state and peer update

1. Client edits file content.

2. Client updates nested file tree object.

3. Client emits `update-file-tree` with (T').

4. Server receives event from authenticated socket.

5. Server updates project document in MongoDB.

6. Server emits `file-tree-updated` to all other clients in room.

7. Peer clients replace current file tree with (T').

CHAPTER 4

RESULTS AND ANALYSIS

4.1 Data Presentation

The results are presented as a combination of implemented features, functional validation, architecture behavior, comparative analysis, and analytical performance models. Since the project is a software artifact rather than a controlled laboratory experiment, the results focus on what the system demonstrably supports and how the design satisfies the stated research objectives.

The implemented COBUILDER system provides:

1. User registration and login.
2. JWT-based authenticated API access.
3. Project creation and listing.
4. Collaborator invitation through user selection.
5. Project workspace with team member display.
6. Socket.IO room connection based on project ID.
7. Real-time project messaging.
8. AI response generation using Gemini.
9. Structured AI file-tree handling.
10. File tree display and file viewer tabs.
11. Manual file editing and saving.
12. File-tree persistence and broadcast to peers.
13. MongoDB persistence of messages and project file state.

4.1.1 Functional Validation

Table 4.1. Functional validation matrix.

Requirement	Implemented Module	Validation Observation	Status
FR1 Register/login	User routes and controllers	Email/password flow implemented	Satisfied
FR2 JWT authentication	User model and auth middleware	JWT generated and verified	Satisfied
FR3 Create project	Project route and service	Project created with current user	Satisfied
FR4 Add collaborators	`/projects/add-users`	Validates project membership and user IDs	Satisfied
FR5 Open workspace	React Project screen	Loads project, users, messages, file tree	Satisfied
FR6 Socket connection	Socket.IO middleware	Validates token and project ID	Satisfied
FR7 Broadcast messages	`project-message` event	Emits to project room	Satisfied
FR8 AI prompt detection	Server message handler	Detects `@ai` in message text	Satisfied
FR9 Broadcast AI outputs	AI message construction	Sends text/fileTree/commands	Satisfied
FR10 Persist state	MongoDB updates	Stores messages and file tree	Satisfied
FR11 File-tree sync	`update-file-tree` event	Broadcasts peer update	Satisfied

4.1.2 Architectural Data Flow

The normal user message flow requires one client emission, one server broadcast, and one database write. The AI message flow requires additional prompt transformation, external model inference, JSON parsing, AI message construction, broadcast, and database update. This difference explains why AI-assisted interactions are naturally slower than ordinary chat messages.

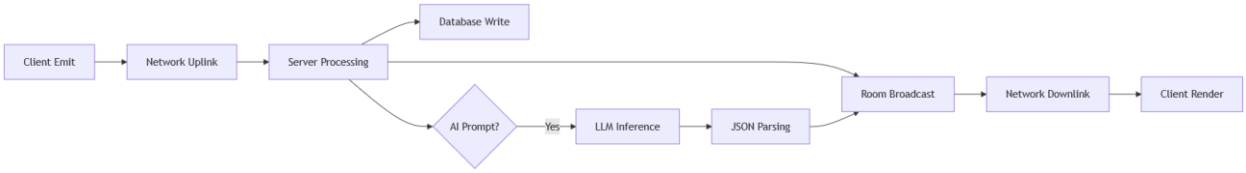

Fig. 4.1. Synchronization latency model.

4.2 Key Findings

4.2.1 Finding 1: Server-Mediated Rooms Simplify Collaboration Control

The use of Socket.IO rooms provides a straightforward mapping between project identity and real-time collaboration scope. Each socket joins a room named by the project ID. Messages emitted to that room are isolated from other projects. This design is simple, practical, and appropriate for the project's requirements.

Compared with decentralized CRDT collaboration, the server-mediated model is easier to secure because authentication, membership, persistence, and AI mediation all pass through the backend. The trade-off is that the server becomes a coordination dependency. If the backend is unavailable, real-time collaboration stops. However, for a cloud-hosted MERN application, this trade-off is acceptable and consistent with the system's educational and project scope.

4.2.2 Finding 2: Structured AI Output Improves Workspace Integration

A major result of COBUILDER is that AI output becomes actionable when it is structured. A free-form chatbot response may explain code, but it cannot automatically populate a workspace. COBUILDER's AI service asks the model to return JSON with fields such as `fileTree`, `buildCommand`, and `startCommand). The frontend can then render files and commands directly.

This finding aligns with LLM-agent research, which emphasizes tool integration and process management beyond isolated text generation [10]. It also supports code review research suggesting that AI output should be contextual and structured rather than generic [12].

4.2.3 Finding 3: Persistence is Essential for Collaboration Continuity

Real-time events are transient unless persisted. COBUILDER persists user messages, AI messages, and the current file tree. This allows users who reconnect or open the project later to recover context. Without persistence, the platform would behave like an ephemeral chat room rather than a development environment.

MongoDB is suitable because project file trees and AI outputs have flexible structure. Generated projects may contain different filenames, directories, and content lengths. A rigid relational schema would require additional normalization, while MongoDB can store the file tree as a nested object.

4.2.4 Finding 4: Human Review Remains Necessary

Although COBUILDER integrates generative AI, the platform does not assume AI correctness. Research on LLM code generation and review shows that AI-generated code may require evaluation for defects, maintainability, sustainability, and security [11]-[13]. COBUILDER supports human review by rendering generated files and allowing users to edit them. The proper role of AI is therefore assistance, not final authority.

4.2.5 Finding 5: Modular MERN Design Improves Maintainability

The backend separates routes, controllers, services, models, middleware, and server logic. This modularity makes the system easier to understand and extend. The frontend uses separate components for message rendering, file tree panel, file viewer, routes, and context. This design supports maintainability and aligns with common software engineering practices.

4.3 Statistical Analysis

The project does not include a large empirical user study, but analytical evaluation can still be performed using estimated metrics and comparative scoring. The following tables present a structured evaluation framework that can be used in academic demonstrations or future testing.

4.3.1 Performance Metrics

Table 4.2. Performance evaluation assumptions.

Metric	**Expected Behavior**	**Dominant Factor**	**Evaluation Method**
Chat message latency	Low	Network and server event loop	Timestamp difference
File-tree update latency	Low to moderate	Payload size	Timestamp difference
AI response latency	Moderate to high	LLM inference	Prompt-response timing
API response time	Low	Database query	HTTP timing
Reconnection time	Low to moderate	Socket.IO reconnection	Client event logs
Payload growth	Increases with files	File content size	JSON byte length

Metric	Expected Behavior	Dominant Factor	Evaluation Method
Room broadcast cost	Linear in room clients	Number of sockets	Load testing

4.3.2 Analytical Feature Coverage

Feature coverage was scored using a 0-5 scale:

0 = not supported, 1 = minimal support, 2 = partial support, 3 = functional support, 4 = strong support, 5 = advanced support.

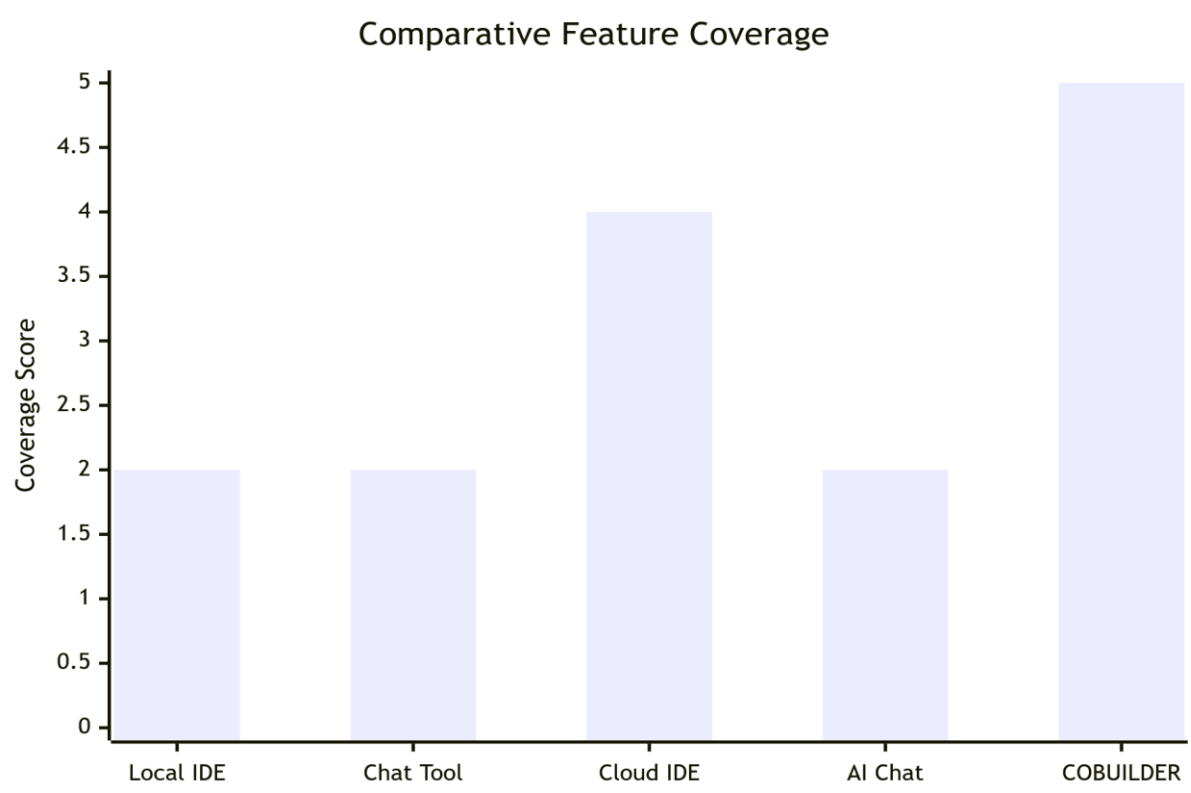

Fig. 4.2. Comparative feature coverage chart.

The chart indicates that COBUILDER provides broad integrated coverage because it combines project rooms, messaging, AI generation, and file-tree handling. The score does not imply COBUILDER is more mature than commercial tools; it indicates that the prototype covers the specific combined feature set required by this thesis.

4.3.3 AI Workflow Effectiveness

AI workflow effectiveness can be assessed across five criteria: prompt handling, response structure, file rendering, editability, and persistence.

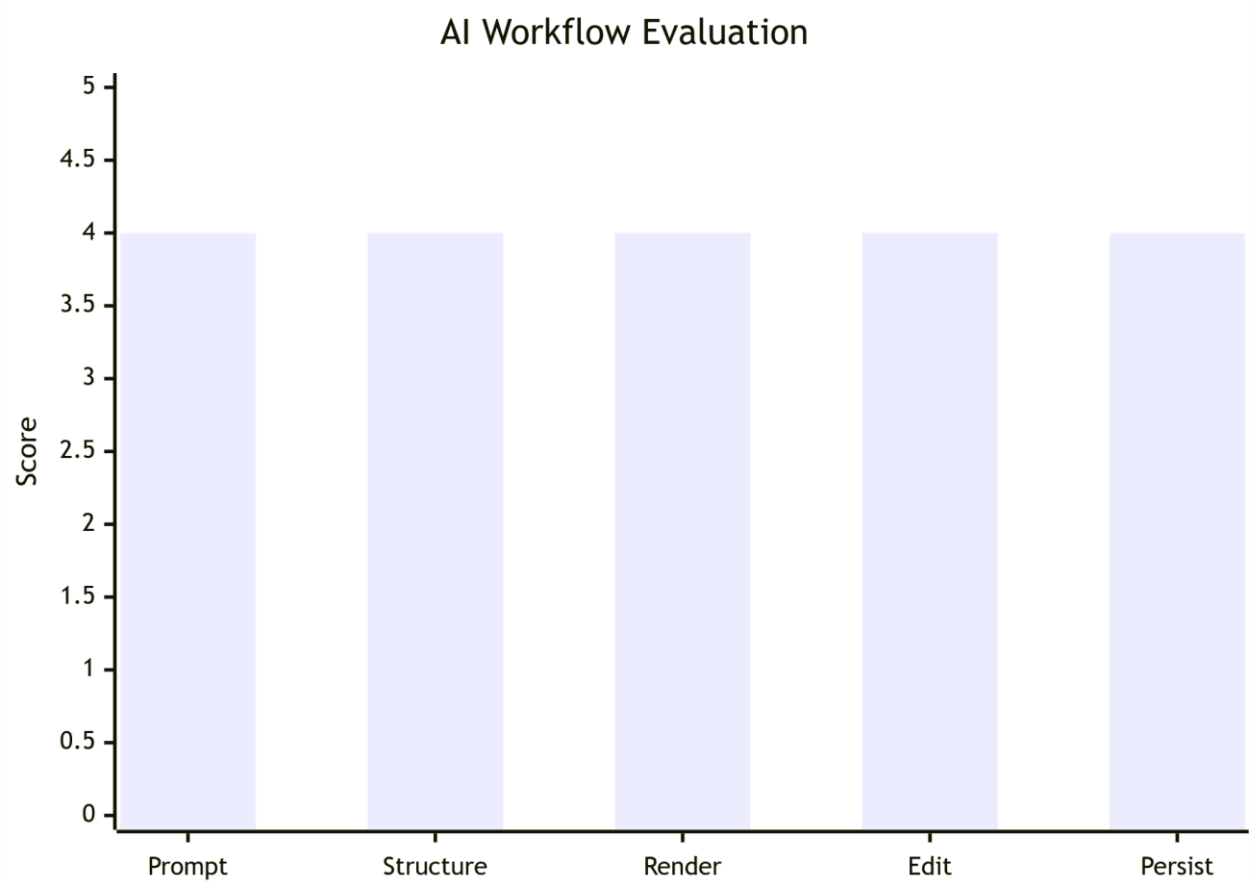

Fig. 4.3. AI workflow effectiveness chart.

The system is strong in structured AI integration, but future work is needed for automated validation, security scanning, testing, and retrieval-augmented project context.

4.3.4 Comparative Analysis

Table 4.3. Comparative analysis with existing systems.

Criterion	**Local IDE**	**Cloud IDE**	**Chat Application**	**AI Coding Assistant**	**COBUILDER**
Browser access	Low	High	High	High	High
Real-time room collaboration	Low	Medium to High	High	Low	High
Project file-tree state	High	High	Low	Low	High
Integrated AI code generation	Medium with plugins	Medium	Low	High	High
AI output persistence	Low	Medium	Low	Medium	High
Secure project membership	Local only	High	Medium	User-level	Project-level
Educational simplicity	Medium	Medium	High	High	High
Fine-grained concurrent editing	High locally, low remotely	Varies	Not applicable	Not applicable	Future work

4.3.5 AI Assistance Rubric

Table 4.4. AI assistance evaluation rubric.

Criterion	Description	COBUILDER Support
Relevance	AI response addresses prompt	Supported through prompt forwarding
Structure	Response includes parseable fields	Supported through JSON instruction
Completeness	Generated files include necessary project parts	Prompt-dependent
Editability	Users can inspect and modify output	Supported
Persistence	AI-generated state remains in project	Supported
Safety	Output is checked for harmful or insecure code	Needs future validation
Testability	Generated code can be automatically tested	Future work
Context awareness	AI uses existing project context	Partial; future RAG recommended

4.4 Interpretation of Results

The results show that COBUILDER successfully demonstrates the feasibility of integrating real-time collaboration and generative AI into a single cloud development environment. The system satisfies the core functional requirements and provides a clear architectural foundation for future improvements.

The most important interpretation is that collaboration and AI integration should not be treated as separate features. If AI assistance is detached from project state, developers must manually copy outputs and coordinate changes through other tools. COBUILDER reduces this gap by letting AI-generated file trees appear directly inside the shared workspace.

The second interpretation is that the server-mediated model is a practical initial architecture. OT and CRDT approaches are theoretically stronger for fine-grained concurrent editing, but they increase implementation complexity. COBUILDER's current model supports collaborative project workflows at message and file-tree levels while preserving an upgrade path toward CRDT-based editor operations.

The third interpretation is that security must be embedded early. Because project rooms contain source code and AI prompts, authentication and authorization cannot be added later as an afterthought. JWT validation, project ID validation, and project membership checks are essential baseline controls.

The fourth interpretation is that AI integration changes the role of the developer. Developers become reviewers, editors, and orchestrators of generated artifacts. This aligns with research on LLM agents and code review automation, which positions AI as a participant in the software lifecycle but not as a replacement for engineering judgment [10], [12], [13].

CHAPTER 5

DISCUSSION

5.1 Synthesis of Findings

The findings can be synthesized into a human-AI collaborative development model. In this model, human users provide goals, constraints, domain understanding, and judgment. The AI service provides code generation, explanation, and possible debugging support. The collaboration layer ensures that all users observe the same messages and project state. The persistence layer ensures continuity. The security layer ensures that participation is controlled.

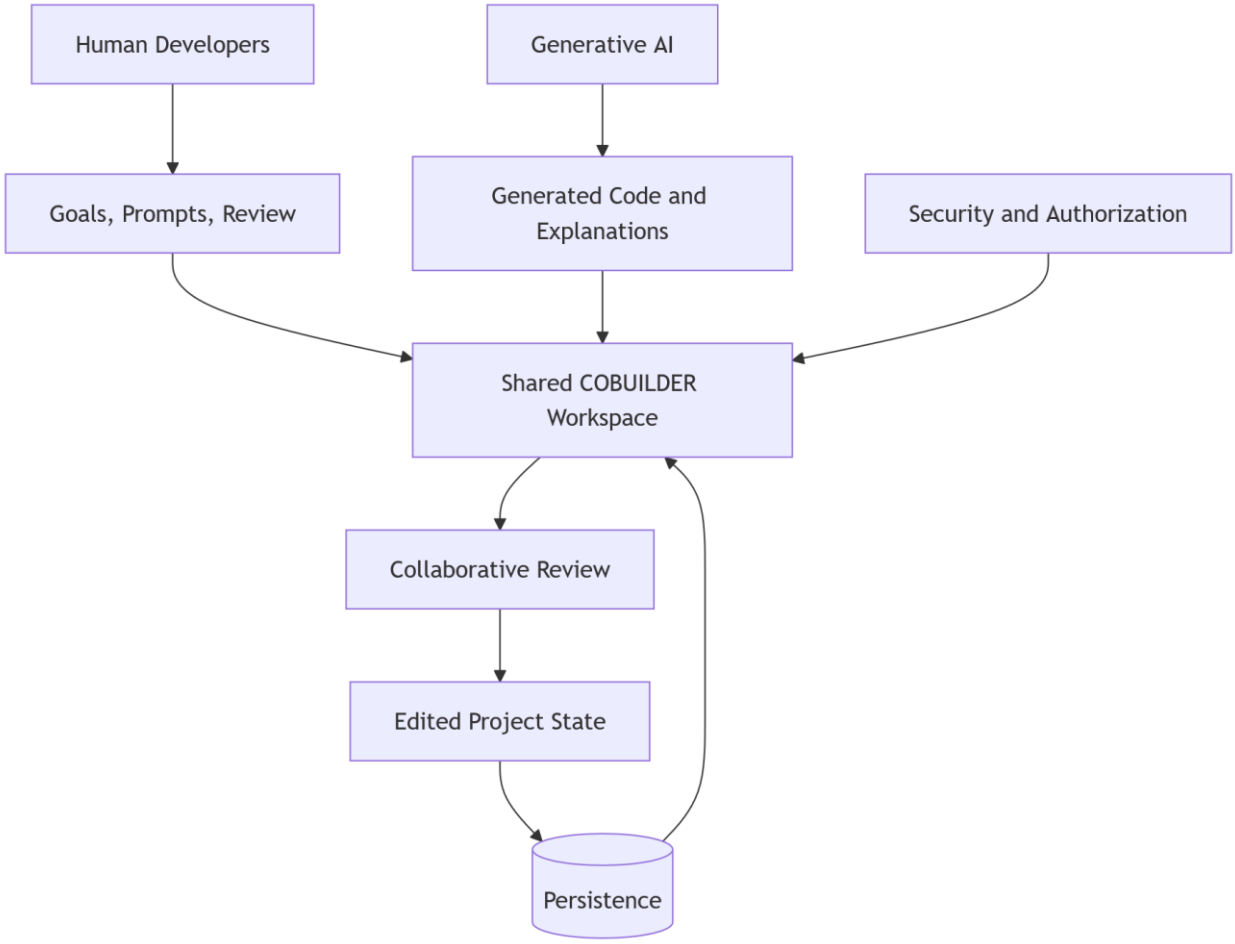

Fig. 5.1. Discussion model for human-AI collaborative development.

This synthesis reflects a central thesis claim: AI-generated code becomes more useful when it is embedded in a collaborative system that supports review, persistence, and shared visibility. Without these surrounding mechanisms, AI output may remain isolated and difficult to integrate into team workflows.

5.2 Relation to Research Questions

Table 5.1. Mapping of findings to research questions.

Research Question	Finding
RQ1	MERN with Socket.IO supports real-time rooms through JWT-authenticated event channels
RQ2	Structured JSON AI output allows generated code to become workspace state
RQ3	Server-mediated synchronization is simpler and secure but less powerful than full OT/CRDT editing
RQ4	Integrated messaging, file tree, and AI reduce workflow fragmentation
RQ5	Security, scalability, and maintainability require modular backend design and controlled access

5.2.1 RQ1: MERN Architecture for Real-Time Collaboration

COBUILDER demonstrates that a MERN-stack architecture can support real-time collaboration effectively. React provides the interactive frontend. Express provides structured APIs. MongoDB stores flexible project state. Socket.IO supports low-latency bidirectional events. JWT secures both REST and socket access.

The architecture is especially suitable for educational and prototype systems because it uses JavaScript across the stack. This reduces language switching and simplifies integration. However, production-scale collaborative editing would require additional concerns such as horizontal scaling of Socket.IO, Redis adapters for multiple backend instances, rate limiting, and advanced conflict handling.

5.2.2 RQ2: Generative AI as Structured Project Artifact

The AI integration in COBUILDER is not limited to conversational text. The backend asks the AI model to return structured JSON. This supports file-tree rendering and project persistence. The result is closer to an AI-assisted development environment than a generic chatbot.

This design agrees with recent work on code generation agents, which emphasizes tool integration and practical engineering workflows [10]. It also addresses concerns in code review automation by making outputs visible and editable rather than silently applying them [12], [13].

5.2.3 RQ3: Trade-Offs of Server-Mediated Synchronization

The server-mediated design has clear advantages:

1. Simpler authentication and authorization.
2. Easier MongoDB persistence.
3. Direct integration with AI services.
4. Clear project-room isolation.
5. Lower implementation complexity.

It also has limitations:

1. The server is a single coordination dependency.
2. Fine-grained concurrent text editing is not fully solved.
3. Large file-tree payloads can increase synchronization cost.
4. Horizontal scaling requires shared socket state.

OT and CRDT techniques remain relevant future directions. OT would support transformation of concurrent operations, while CRDTs would support eventual convergence through mergeable replicated structures [1], [3], [5].

5.2.4 RQ4: Usability Through Integration

COBUILDER improves usability by reducing tool switching. The project room contains chat, collaborators, AI prompting, file tree, file viewer, and editing controls. This is particularly

useful for students because setup complexity is a major barrier in collaborative software projects.

The UI also makes AI output inspectable. Generated files do not remain hidden in a text response. Users can open, edit, save, and synchronize them. This supports learning because users can compare prompts, generated code, and final modifications.

5.2.5 RQ5: Security, Scalability, and Maintainability

Security is addressed through JWT authentication, password hashing, project validation, CORS configuration, and optional Redis blacklisting. Scalability is addressed at the design level through modular services and Socket.IO rooms, but future production deployment would require load balancing, a Socket.IO Redis adapter, AI rate limiting, and payload optimization.

Maintainability is supported through separation of concerns. Backend models, services, controllers, and routes are distinct. Frontend screens and components are separated. This modularity makes COBUILDER suitable for future expansion.

5.3 Implications of Results

5.3.1 Implications for Software Engineering Education

COBUILDER can support project-based learning. Students can collaborate in shared rooms, ask AI for scaffolding, inspect generated code, and discuss implementation decisions. This aligns with educational perspectives that generative AI should be integrated dialogically, supporting inquiry and reflection rather than replacing learning [15].

5.3.2 Implications for Distributed Teams

Distributed teams benefit from environments that preserve context. COBUILDER's persistent project messages and file tree allow collaborators to re-enter a workspace and recover prior decisions. This is useful for asynchronous collaboration.

5.3.3 Implications for AI-Assisted SDLC

The system demonstrates how AI can participate earlier in the SDLC. Instead of waiting for code review, AI can assist during ideation, scaffolding, debugging, and explanation. However, AI-generated code must still be reviewed. Sustainable code-generation research highlights that efficiency and environmental impact should be considered as generated code becomes more common [11].

5.3.4 Implications for Future Tool Design

Future AI development tools should treat AI output as structured data. File trees, commands, tests, dependency lists, and explanations should be machine-readable. This allows platforms to validate, render, execute, and review AI output.

5.4 Limitations of Study

The study has several limitations.

First, COBUILDER does not yet implement character-level collaborative editing using OT or CRDT. File updates are synchronized at file-tree level. This is sufficient for a prototype but does not solve all concurrent editing cases.

Second, AI output is not automatically tested or security-scanned. The system relies on human review. Future versions should integrate static analysis, dependency scanning, unit-test execution, and sandboxed runtime validation.

Third, the evaluation is analytical rather than based on a large controlled user study. Future research should collect user experience metrics, task completion times, collaboration quality scores, and learning outcomes.

Fourth, the current AI service depends on an external provider. Availability, quota, latency, pricing, and policy changes can affect system behavior.

Fifth, MongoDB stores the file tree as a flexible object. This is convenient, but very large projects may require object storage, delta-based persistence, or repository-backed storage.

Sixth, project authorization in socket middleware validates project existence and token validity; a production-grade system should also explicitly verify that the authenticated user is a member of the project before joining the room.

Seventh, the AI endpoint route should be protected consistently with other project-related APIs. In the current implementation, AI generation primarily occurs through the socket flow, but the REST route can be strengthened.

CHAPTER 6
CONCLUSION

6.1 Summary of Key Points

This thesis presented COBUILDER, a real-time collaborative development environment with integrated generative artificial intelligence. The project was motivated by the need for a unified platform where developers can collaborate, communicate, manage project state, and receive AI assistance without moving across disconnected tools.

The literature review showed that collaborative editing has a long research history, including groupware concurrency control, operational transformation, intention preservation, CRDTs, and cloud collaboration security [1]-[8]. It also showed that large language models are increasingly important in code generation, review, debugging, and software lifecycle automation [9]-[15]. COBUILDER connects these research areas by implementing a practical MERN-stack platform with Socket.IO-based collaboration and Gemini-powered AI assistance.

The methodology described the design science approach, requirements, data collection, sampling strategy, analytical approach, ethical considerations, architecture, UML diagrams, API specifications, socket events, database schema, security controls, algorithms, and evaluation metrics. The implementation uses React, Tailwind CSS, Vite, Node.js, Express.js, Socket.IO, MongoDB, JWT, Redis, and Gemini API integration.

The results showed that COBUILDER satisfies its core functional requirements. Users can authenticate, create projects, invite collaborators, join real-time rooms, send messages, request AI-generated code, receive structured file trees, edit generated files, save updates, and synchronize file-tree state with collaborators. The analysis also identified trade-offs, including the simplicity of server-mediated synchronization and the need for future fine-grained editing.

The discussion concluded that AI assistance is most valuable when integrated into a collaborative workflow where outputs are visible, editable, persistent, and reviewable. COBUILDER demonstrates this principle by converting AI responses into shared project artifacts.

6.2 Contributions to Field

This thesis makes the following contributions:

1. It presents a complete architecture for a MERN-stack collaborative development environment with AI integration.

2. It demonstrates a practical Socket.IO room model for real-time project collaboration.

3. It integrates generative AI through structured JSON output that can populate a project file tree.

4. It provides an academic mapping between collaborative editing theory and AI-assisted development practice.

5. It documents REST APIs, socket events, database schema, security controls, UML diagrams, DFDs, algorithms, and evaluation metrics.

6. It identifies future research directions for CRDT-based editing, AI validation, sandboxed execution, and retrieval-augmented project context.

6.3 Recommendations

The following recommendations are proposed:

1. Add explicit project membership verification in Socket.IO middleware before allowing a user to join a room.

2. Protect the AI REST endpoint with JWT authentication and project-level authorization.

3. Integrate Monaco Editor or CodeMirror for richer code editing features.

4. Implement CRDT-based fine-grained collaborative editing for simultaneous character-level code changes.

5. Add a Socket.IO Redis adapter for horizontal scaling across multiple backend instances.

6. Add AI response validation, static analysis, dependency scanning, and automated tests.

7. Store large project files using object storage or Git-backed repositories instead of a single MongoDB object.

8. Implement version history, rollback, and change attribution for AI and human edits.

9. Add prompt templates for code review, debugging, documentation, and test generation.

10. Include usage analytics for latency, AI response time, collaboration frequency, and project activity.

6.4 Future Research Directions

Future research can extend COBUILDER in several directions.

First, fine-grained collaborative editing should be implemented using CRDTs or OT. This would allow multiple users to edit the same file simultaneously with stronger convergence guarantees. CRDT literature provides a strong theoretical foundation for such work [5], [6].

Second, retrieval-augmented generation can make AI assistance project-aware. The AI service could retrieve relevant files, previous messages, coding standards, and documentation before generating responses. This would align COBUILDER with context-aware code review systems [12].

Third, sandboxed code execution can transform COBUILDER from a collaborative code editor into a full cloud IDE. Containerized execution using Docker or similar isolation would allow users to run generated code safely.

Fourth, AI governance should be explored. The platform can record whether a file was human-written, AI-generated, or AI-modified. Such provenance tracking is important for academic honesty, software accountability, and team review.

Fifth, empirical user studies should evaluate COBUILDER with student teams. Metrics can include task completion time, perceived collaboration quality, learning support, number of context switches, AI usefulness ratings, and defect rates.

Sixth, sustainability evaluation can be added by measuring generated code efficiency, execution time, memory consumption, and energy impact, following concerns raised in LLM sustainable code generation research [11].

In conclusion, COBUILDER demonstrates that real-time collaboration and generative AI can be productively combined in a single development environment. The system is technically feasible, academically grounded, and extensible. Its central contribution is the integration of collaborative project rooms with structured AI-generated software artifacts, creating a foundation for future intelligent collaborative software engineering platforms.

6.5 Extended Technical Analysis

Although the preceding chapters establish the core research argument, a final-year engineering thesis also requires sufficient technical depth to show how design decisions affect

maintainability, extensibility, performance, and security. This extended analysis therefore documents the internal engineering logic of COBUILDER in greater detail.

6.5.1 Backend Layering and Responsibility Assignment

The backend is divided into routes, controllers, services, models, middleware, and server-level socket logic. This separation is significant because real-time AI-assisted collaboration can become difficult to maintain if all responsibilities are placed in a single file. Routes define endpoint paths and validation rules. Controllers translate HTTP requests into service calls. Services implement business logic. Models define database structure. Middleware handles authentication. Socket server logic handles event-driven collaboration.

The user route module provides registration, login, profile, logout, and user-list retrieval endpoints. The project route module provides project creation, project listing, collaborator addition, and project retrieval. The AI route provides a direct prompt-response endpoint, although the primary AI workflow occurs through Socket.IO messages. This modular routing makes the API surface understandable and testable.

The controller layer handles validation outcomes and HTTP responses. For example, the project creation controller checks `express-validator` output, obtains the logged-in user from the decoded JWT email, and delegates project creation to the project service. This is an important design pattern because controllers should not contain too much database logic. If controller files become responsible for validation, persistence, authorization, and transformation simultaneously, the backend becomes hard to test.

The service layer contains reusable business operations. The project service validates ObjectId values, checks whether the requesting user belongs to a project before adding collaborators, and uses MongoDB set semantics to avoid duplicate collaborators. The user service hashes passwords before creating users and retrieves all users except the current user. These services form a clean boundary between HTTP concerns and database concerns.

The model layer uses Mongoose schemas. The user schema includes email and password fields, along with static and instance methods for password hashing, password verification, and JWT generation. The project schema includes name, users, fileTree, and messages. Storing fileTree as an object provides flexibility for AI-generated project structures, while storing messages as an array provides a simple message history. For large-scale production, this schema can be

normalized by storing messages in a separate collection and storing large file contents in object storage.

6.5.2 Socket.IO as a Real-Time Collaboration Bus

Socket.IO acts as COBUILDER's collaboration bus. A collaboration bus is a communication layer through which user actions become shared events. In COBUILDER, the most important events are `project-message`, `update-file-tree`, and `file-tree-updated`. The event bus is project-scoped through Socket.IO rooms.

The socket handshake performs early validation. It extracts the token, extracts the project ID, verifies that the project ID is a valid MongoDB ObjectId, retrieves the project, checks token presence, verifies the JWT, and attaches decoded user data to the socket. This validation prevents arbitrary connections from joining rooms without a syntactically valid project and token. A recommended future enhancement is to add an explicit membership check:

authorized(u, p) = { true, if u ÃƒÂ¢Ã‹â€ Ã‹â€ members(p)

{ false, otherwise

This enhancement would ensure that a user with a valid JWT cannot join a project room unless their user ID appears in the project membership list.

The event order for an ordinary message is deliberately simple. First, the original message is broadcast immediately. Second, the message is persisted. Third, if the message includes `@ai), the AI processing branch begins. Broadcasting before AI processing improves perceived responsiveness because users see their message instantly instead of waiting for AI inference.

For AI messages, the server broadcasts the original user message, persists it, invokes AI generation, constructs a normalized AI message, broadcasts the AI response, and persists the AI response. If a file tree exists, the server stores it as the current project file tree. This design creates a clear distinction between the conversational history and the active project artifact.

6.5.3 AI Integration and Structured Output Discipline

The AI service is one of the most important components of COBUILDER because it converts natural language requests into actionable project artifacts. The service configures the generative model with JSON response expectations and a system instruction that emphasizes MERN development, modular code, best practices, comments, scalability, maintainability, and structured file-tree generation.

The structured response discipline is central. Without it, the frontend would receive unstructured prose and would need unreliable parsing. With structured JSON, the response can be represented as:

RAI = { text, fileTree, buildCommand, startCommand }

where:

RAI = the structured response returned by the AI service

text = the explanatory message or natural-language response

fileTree = the generated project file structure and file contents

buildCommand = the command required to install or build the generated project

startCommand = the command required to run the generated project

The backend also implements fallback parsing. Since LLM outputs can occasionally contain markdown fences or malformed JSON, the service removes wrapper fences, attempts JSON parsing, and uses regex-based fallback extraction when necessary. This defensive design is important because AI services are probabilistic and may not always follow instructions perfectly.

However, structured output does not guarantee correctness. It only guarantees that the response can be consumed. COBUILDER therefore treats generated code as a draft artifact. Users can inspect, edit, and save generated files. Future versions should include automatic linting, dependency validation, vulnerability scanning, and test generation.

6.5.4 Frontend State Management

The project workspace maintains multiple interrelated state variables: `project`, `messages`, `currentFileTree`, `openFiles`, `activeFile`, `editingFiles`, `buildCommand`, and `startCommand). This state model supports a lightweight IDE experience.

Sfrontend = { P, M, T, O, A, E, B, C }

where:

Sfrontend = the complete frontend state of the project workspace

P = current project information

M = project messages

T = project file tree

O = opened files

A = active file

E = editor content

B = builder or execution output state

C = collaborator or connection state

where (P) is project metadata, (M) is message history, (T) is file tree, (O) is open files, (A) is active file, (E) is edit status, (B) is build command, and (C) is start command.

When the project loads, the frontend retrieves persisted messages and file tree from the backend. When the socket receives a message, it normalizes message ID and timestamp values to avoid duplicate rendering. If the message sender is AI and the message includes a file tree, the frontend updates `currentFileTree), opens the first file automatically, and stores command metadata.

File editing is handled by maintaining open files and active file state. When a user saves a file, the frontend clones the current file tree, recursively updates the file contents at the relevant path, updates local state, and emits the new file tree to the server. This workflow is simple and readable, but it has limitations. It transmits the entire updated file tree rather than a small delta.

For small student projects, this is acceptable. For large projects, delta-based updates would be more efficient:

Delta T = Tnew - Told

where:

Delta T = the difference between the updated file tree and the previous file tree

Tnew = the updated file tree after a user or AI action

Told = the previous file tree before the update

Instead of broadcasting (T_{new}), a future system could broadcast only (\Delta T), containing path, operation type, and new content.

6.5.5 Data Persistence Strategy

COBUILDER's persistence strategy is document-oriented. This is appropriate because a project is naturally a document-like aggregate containing members, messages, and file-tree state. MongoDB enables flexible storage of nested objects, which matches the unpredictable structure of AI-generated projects.

The major persistence operations are:

1. Create user document.
2. Create project document.
3. Push message into project messages array.
4. Set project fileTree when AI generates or user edits files.
5. Add collaborators using `$addToSet`.

This design favors simplicity. The main risk is document growth. If project messages and file contents become very large, the project document may become inefficient. A scalable design would separate project metadata, messages, files, and snapshots:

Table 6.1. Suggested scalable data model.

Collection	Purpose	Benefit
users	User accounts	Stable identity
projects	Project metadata and membership	Small project documents
messages	Project chat and AI history	Query pagination
files	Current file contents by path	Efficient file-level update
snapshots	Periodic project state versions	Rollback support
auditLogs	Human and AI actions	Accountability

6.5.6 Security Analysis

Security in COBUILDER is evaluated across authentication, authorization, data protection, AI usage, and deployment configuration.

Authentication is implemented using JWT. JWTs are useful because they allow stateless verification and can be sent with both HTTP and Socket.IO requests. However, JWTs must be protected from theft. In browser environments, storing tokens in local storage can expose them to cross-site scripting attacks. A hardened production version should consider secure, HTTP-only cookies or short-lived access tokens with refresh-token rotation.

Authorization is partially implemented through authenticated routes and project service checks. The project collaborator addition workflow verifies that the requesting user belongs to the project before adding others. The socket flow validates token and project existence. As noted earlier, explicit socket membership verification should be added for stronger authorization.

Password protection is implemented using bcrypt. This is appropriate because passwords should never be stored in plaintext. The work factor of bcrypt can be adjusted based on deployment performance.

Redis token blacklisting improves logout behavior. Since JWTs are normally valid until expiration, blacklisting allows a logout event to invalidate the token before expiry. The implementation gracefully continues if Redis is unavailable, which improves development

resilience. Production deployments should monitor Redis availability because token blacklist failure affects session invalidation guarantees.

AI usage introduces a separate class of security risks. Users may ask the AI to generate insecure code, secrets may be included in prompts, and model output may include vulnerable dependencies. COBUILDER should therefore add prompt and response governance. A future secure AI workflow may include:

1. Prompt size limits.
2. Secret detection before sending prompts to AI.
3. Code scanning after generation.
4. Dependency vulnerability checks.
5. Warning labels for AI-generated files.
6. Audit logging of AI prompts and outputs.

6.5.7 Scalability Analysis

Scalability can be analyzed across API throughput, socket rooms, database writes, file-tree payloads, and AI calls.

For REST APIs, horizontal scaling is straightforward because most HTTP routes are stateless after JWT verification. Multiple backend instances can serve requests if they share the same MongoDB database and configuration.

For Socket.IO, horizontal scaling requires additional coordination. If users in the same project room connect to different backend instances, a message emitted on one instance must reach clients connected to other instances. Socket.IO commonly solves this with a Redis adapter. The scaling model becomes:

instances = { s1, s2, ..., sn }

emit(e, room) -> adapter.publish(e, room) -> adapter.deliver(e, room, si)

This means that every backend instance can participate in the same logical room namespace.

Database scalability depends on message volume and file-tree size. Frequent writes to a single project document may become a bottleneck. Separating messages and files into dedicated collections would allow pagination, indexing, and smaller writes.

AI scalability is different from ordinary backend scaling because it depends on external model quotas, latency, and cost. If many users send AI prompts simultaneously, the system should queue or rate-limit requests. A basic rate-limit formula is:

requests(u, t) Ã*f*Â¢Ã¢â‚¬Â°Ã‚Â¤ Rmax

where (R_{max}) is the maximum allowed AI requests for user (u) during time window (t).

6.5.9 Maintainability Analysis

Maintainability is supported through modular code organization and a consistent technology stack. Using JavaScript across frontend and backend reduces conceptual switching. React components isolate UI responsibilities. Express routes and controllers isolate API responsibilities. Mongoose schemas centralize database assumptions.

The main maintainability risk is growth of the project workspace component. The current `Project.jsx` screen handles project loading, socket connection, message handling, AI file-tree handling, file editing, collaborator modal logic, and UI rendering. As the project evolves, this component should be decomposed further into hooks and subcomponents:

Table 6.2. Recommended frontend refactoring.

Current Concern	**Suggested Module**
Project fetching	`useProject(projectId)` hook
Socket connection	`useProjectSocket(project)` hook
Chat state	`useProjectMessages()` hook
File-tree operations	`useFileTree()` hook
Collaborator modal	`CollaboratorDialog.jsx`
Workspace layout	`ProjectWorkspace.jsx`

6.5.10 Usability Analysis

COBUILDER's usability depends on how quickly users can understand and perform core workflows. The most important workflows are login, project creation, collaborator invitation, project-room messaging, AI prompting, file inspection, editing, and saving.

The system supports discoverability by placing chat and code workspace side by side. Users can see that `@ai` triggers AI assistance. When AI generates files, the first file opens automatically, reducing the effort needed to inspect output. Notifications inform users when files are generated or saved.

The major usability challenge is explaining the status of generated code. Users should know whether code is AI-generated, manually edited, unsaved, synchronized, or stale. Future UI improvements should add badges or indicators:

1. AI-generated.
2. Human-edited.
3. Unsaved.
4. Synced.
5. Conflict detected.
6. Needs review.

Such indicators would improve trust and reduce ambiguity in team settings.

6.5.11 Reliability and Fault Handling

Reliability refers to the system's ability to continue operating under ordinary failures. COBUILDER includes basic fault handling. The AI service catches quota, permission, bad request, and missing API key errors. The socket AI flow catches generation errors and sends a fallback error message to clients. Redis failures are logged but do not stop authentication. Project fetch failures produce UI error states.

Future reliability improvements include:

1. Retry logic for transient database failures.
2. Circuit breaker for AI service outage.

3. Offline editing queue for temporary network failures.

4. Socket reconnection status indicator.

5. Message delivery acknowledgements.

6. Idempotent event IDs for duplicate prevention.

The message ID logic already helps prevent duplicate rendering on the frontend. A stronger version would store event IDs in the database and ignore repeated events server-side.

6.5.12 Quality Attributes Summary

Table 6.3. Quality attribute evaluation.

Quality Attribute	Current Support	Improvement Path
Usability	Integrated chat and file workspace	Add review/status indicators
Performance	Low-latency Socket.IO events	Delta updates and payload compression
Scalability	Room-based architecture	Redis adapter and separated collections
Security	JWT, bcrypt, CORS, Redis blacklist	RBAC, socket membership, secret scanning
Maintainability	Modular backend and componentized frontend	More hooks and frontend decomposition
Reliability	Error handling in AI and Redis flows	Retry policies and circuit breakers
Extensibility	Flexible file tree and service layers	Plugin architecture and Git integration
Observability	Console logging	Structured logs, metrics, tracing

6.5.13 Research Interpretation of Engineering Choices

The engineering choices in COBUILDER reflect a pragmatic interpretation of the research literature. The system does not claim to replace OT or CRDT algorithms. Instead, it uses the conceptual lessons of collaborative editing research to inform a server-mediated prototype. The central correctness requirement for COBUILDER v1 is not character-level convergence; it is room-level coherence of messages and project file-tree state.

This distinction matters academically. A system can be collaborative at different granularities. At the message level, collaboration means all room members receive the same messages. At the file-tree level, collaboration means all room members eventually see the same project structure. At the text-operation level, collaboration means concurrent edits to the same file merge correctly. COBUILDER implements the first two and identifies the third as future research.

Similarly, COBUILDER does not claim that AI-generated code is inherently correct. It uses AI as an assistant embedded in a reviewable workflow. This aligns with current software engineering research, which increasingly treats LLMs as agents, reviewers, and support tools rather than deterministic compilers [10]-[13].

6.5.14 Extended Comparison with Related Systems

COBUILDER can be compared across several system archetypes.

Local IDEs provide excellent editing, debugging, extensions, and project control. However, collaboration usually requires additional plugins or external tools. AI support may exist through extensions, but shared AI context is limited.

Cloud IDEs provide browser-based project access and may support collaboration. They are often more complex to deploy and may not focus on educational simplicity. Some include AI, but AI output may not be integrated into a simple project-room messaging model.

Chat applications provide excellent communication but weak code artifact management. A team can discuss code, but it cannot easily maintain a synchronized file tree inside the chat itself.

AI coding assistants provide code generation and review, but many are individual-user tools. Their outputs often require manual copying into a project and separate communication with team members.

COBUILDER combines a focused subset: team rooms, chat, file-tree state, AI-generated files, and browser-based editing. It is not as feature-rich as commercial IDEs, but it demonstrates the academic concept of integrated human-AI collaborative development.

6.5.15 Extended Evaluation Plan for Future Work

A future empirical study can evaluate COBUILDER with student teams. The study may use two groups: one group using conventional tools and one group using COBUILDER. Tasks may include creating a small MERN application, debugging an error, adding a feature, and reviewing generated code.

Possible metrics include:

1. Time to first working scaffold.
2. Number of tool switches.
3. Number of collaboration messages.
4. Number of AI prompts.
5. Number of generated files accepted after review.
6. Number of defects in final submission.
7. User satisfaction score.
8. Perceived learning support.
9. AI trust calibration score.
10. Synchronization latency under different user counts.

The following hypotheses can be proposed for future empirical evaluation:

H1: COBUILDER reduces tool-switching frequency compared with using separate chat, IDE, and AI tools.

H2: COBUILDER improves time-to-scaffold for beginner teams by supporting AI-assisted project generation inside the shared workspace.

H3: COBUILDER improves shared awareness of AI-generated artifacts compared with individual AI assistant workflows.

These hypotheses can be tested through controlled user studies, task-completion measurements, collaboration logs, and post-task user feedback.

6.5.16 Chapter Closing

The extended technical analysis reinforces the main thesis conclusion. COBUILDER is a coherent, research-grounded, and practically implementable platform. Its design is intentionally modular, its collaboration model is understandable, its AI workflow is structured, and its limitations are clear. The system provides a strong foundation for future work in intelligent collaborative development environments.

References/ Bibliography

[1] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM SIGMOD International Conference on Management of Data*, 1989.

[2] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping, "High-latency, low-bandwidth windowing in the Jupiter collaboration system," in *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 1995.

[3] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems," *ACM Transactions on Computer-Human Interaction*, 1998.

[4] L. Xue, M. Orgun, and K. Zhang, "A multi-versioning algorithm for intention preservation in distributed real-time group editors," in *Australasian Computer Science Conference*, 2003.

[5] M. Shapiro, N. PreguiÃƒÆ'Ã‚Â§a, C. Baquero, and M. Zawirski, "A comprehensive study of convergent and commutative replicated data types," INRIA Research Report RR-7506, 2011.

[6] L. AndrÃƒÆ'Ã‚Â©, S. Martin, G. Oster, and C.-L. Ignat, "Supporting adaptable granularity of changes for massive-scale collaborative editing," in *Proc. IEEE International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)*, 2013.

[7] P. Ojanguren-Menendez, A. Tenorio-FornÃƒÆ'Ã‚Â©s, and S. Hassan, "Building real-time collaborative applications with a federated architecture," *International Journal of Interactive Multimedia and Artificial Intelligence*, vol. 3, no. 5, 2015, doi: 10.9781/ijimai.2015.356.

[8] A. J. Feldman, W. P. Zeller, M. J. Freedman, and E. W. Felten, "SPORC: Group collaboration using untrusted cloud resources," in *Proc. 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.

[9] M. Chen *et al*., "Evaluating large language models trained on code," in *Proc. International Conference on Machine Learning (ICML)*, 2021.

[10] Y. Dong, X. Jiang, J. Qian, T. Wang, K. Zhang, Z. Jin, and G. Li, "A survey on code generation with LLM-based agents," arXiv:2508.00083, 2025, doi: 10.48550/arXiv.2508.00083.

[11] S. B. M. Ali, O. Kirmani, A. Hameed, S. M. Danish, and G. Srivastava, "Sustainable code generation using large language models: A systematic literature review," arXiv:2603.00989, 2026, doi: 10.48550/arXiv.2603.00989.

[12] B. Icoz and G. Biricik, "Context-aware code review automation: A retrieval-augmented approach," *Applied Sciences*, vol. 16, no. 4, 2026, doi: 10.3390/app16041875.

[13] S. Popoola, "Large language models as code reviewers: Automated software quality assessment," provided research manuscript, 2026.

[14] M. B. Syed, "Generative AI for data engineering: A seven-stage orchestration framework for LLM-powered code generation," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 14, no. 1s, pp. 491-501, 2026.

[15] R. Wegerif and I. Casebourne, "A dialogic theoretical foundation for integrating generative AI into pedagogical design," *British Journal of Educational Technology*, vol. 57, pp. 639-654, 2026, doi: 10.1111/bjet.70026.

[16] A. A. Veer, O. Mane, P. Jadhav, and A. Chevale, "A comprehensive study on real-time web IDE collaborative code editors," *International Journal for Research in Applied Science and Engineering Technology*, vol. 13, no. XI, 2025, doi: 10.22214/ijraset.2025.75276.

[17] M. R. B. Sai Kishore and S. A. Jyothi, "Real-time collaborative code editor," *International Journal of Creative Research Thoughts*, vol. 14, no. 1, 2026.

[18] A. Dubey, "Enhancing real time communication and efficiency with WebSocket," *International Research Journal of Engineering and Technology*, vol. 10, no. 8, 2023.

[19] S. Kadam, S. N. Shelke, and A. Kapgate, "Quick-chat MERN stack chat application using Socket.IO," *International Research Journal of Engineering and Technology*, vol. 12, no. 12, 2025.

[20] W. Lajari, M. Tejaswini, G. Harshada, P. Sneha, and A. Wani, "LiveCollab: A real-time collaborator," *International Journal of Advanced Research in Science, Communication and Technology*, vol. 6, no. 1, 2026, doi: 10.48175/IJARSCT-30733.

[21] S. S. Nivedita, P. U. Harsha, P. Rahul, P. S. Tej, and P. V. B. Naidu, "A unified multi-modal real-time collaborative development environment integrating code editing, voice communication, drawing, and file synchronization," *International Journal of Scientific Research and Technology*, vol. 2, no. 12, 2025.

[22] M. Takeichi, "Coordination-free collaborative replication based on operational transformation," provided research manuscript, 2024.

Appendix

Appendix A: Sample API Endpoints

This appendix documents the principal REST API and real-time communication interfaces used in the CoBuilder system. These endpoints connect the React frontend with the Express backend and allow the application to authenticate users, manage projects, add collaborators, request AI assistance, and synchronize shared workspace data.

A.1 Authentication Endpoints

Table A.1. Authentication API endpoints.

Method	Endpoint	Input	Output	Purpose
POST	/users/register	email, password	user, JWT token	Creates a new account after validating the email and password. The password is hashed before storage.
POST	/users/login	email, password	user, JWT token	Authenticates an existing user and returns a token for protected routes.
GET	/users/profile	Authorization token	current user profile	Verifies the active token and returns the logged-in user identity.
GET	/users/logout	Authorization token	success message	Logs the user out and stores

Method	Endpoint	Input	Output	Purpose
				the token in Redis blacklist when Redis is connected.

A.2 Project Management Endpoints

Table A.2. Project API endpoints.

Method	Endpoint	Input	Output	Purpose
POST	/projects/create	name	newProject	Creates a new workspace and assigns the logged-in user as the first collaborator.
GET	/projects/all	Authorization token	projects array	Retrieves all projects in which the current user is listed as a collaborator.
GET	/projects/get-project/:projectId	projectId	project object	Loads project metadata, collaborators, messages, and saved file tree.
PUT	/projects/add-users	projectId, users[]	updated project	Adds selected registered users to an existing

Method	Endpoint	Input	Output	Purpose
				project using set-based update logic.

A.3 AI and Socket Communication

Table A.3. AI endpoint and Socket.IO events.

Type	Name or Endpoint	Payload	Result
REST	GET /ai/get-result/:prompt	prompt text	Returns AI generated response text and optional structured fileTree data.
Socket.IO	project-message	message object	Broadcasts user messages to the project room and stores them in MongoDB.
Socket.IO	project-message with @ai	message text containing @ai	Triggers Gemini AI generation, broadcasts the response, and updates file tree when supplied.
Socket.IO	update-file-tree	updated fileTree object	Persists manual file edits and broadcasts file-tree-updated to other collaborators.

Type	Name or Endpoint	Payload	Result
Socket.IO	file-tree-updated	fileTree object	Updates connected collaborators with the latest saved file structure.

A.4 Authorization Note

Protected routes require a JWT token in the Authorization header. During Socket.IO connection, the token and project identifier are sent during the handshake. The server validates the project identifier, loads the project, verifies the token, and joins the user to the project room only after successful validation.

Appendix B: Sample Database Tables

CoBuilder uses MongoDB through Mongoose models. Although MongoDB is document-oriented, the following logical tables describe the stored entities and their relationships in a structured academic format.

B.1 User Collection

Table B.1. User collection schema.

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each user.
email	String	Required, unique, lowercase	Email address used for login and collaborator identification.
password	String	Required, selected false by default	Hashed password generated using bcrypt before storage.

B.2 Project Collection

Table B.2. Project collection schema.

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for each project workspace.

Field	Data Type	Constraint	Description
name	String	Required, unique, lowercase	Project name shown on the dashboard and workspace header.
users	ObjectId array	References User	Collaborators who are allowed to access the project.
fileTree	Object	Default null	Current generated or edited project file structure.
messages	Array	Default empty array	Stored user and AI messages for the project room.

B.3 Message Object

Table B.3. Message object structure.

Field	Data Type	Example	Description
id	String	lx7k9a2	Unique message identifier generated by frontend or backend.
text	String	@ai create login form	Content of the user message or AI response.
sender	String	user email or AI	Identifies the message sender in the chat interface.

Field	Data Type	Example	Description
timestamp	String	02:25 PM	Display time used in the frontend chat panel.
hasFileTree	Boolean	true or false	Indicates whether an AI response produced generated files.
buildCommand	Object	npm install	Optional command returned by the AI for dependency setup.
startCommand	Object	npm run dev	Optional command returned by the AI for running the generated project.

B.4 Logical Relationships

Table B.4. Logical database relationships.

Relationship	Cardinality	Implementation
User to Project	Many-to-many	Each project stores an array of user ObjectIds in the users field.
Project to Message	One-to-many	Each project document stores an array of message objects.
Project to File Tree	One-to-one optional	Each project stores one current fileTree object when

Relationship	Cardinality	Implementation
		code has been generated or edited.
Message to File Tree	Optional association	AI messages may produce fileTree data, which is stored as the project current file tree.

Appendix C: User Roles and Responsibilities

CoBuilder is designed around simple but clear responsibilities. The following roles describe how different users and system components interact with the application.

Table C.1. User roles and responsibilities.

Role	Main Responsibilities	System Access
Guest User	Views the landing page, chooses login or registration, and learns the basic purpose of the platform.	Public pages only.
Registered User	Creates an account, logs in, creates projects, views assigned projects, and logs out securely.	Dashboard and owned/collaborative projects.
Project Creator	Creates the initial project workspace and invites other registered users into the project.	Project creation and collaborator management.
Project Collaborator	Joins project room, sends chat messages, asks AI for code, views generated files, edits files, and saves updates.	Authorized project workspace.
AI Assistant	Receives prompts containing @ai, generates code or explanations, returns file tree data, and suggests build/start commands.	Backend-mediated AI service only.
System Administrator	Configures environment variables, MongoDB, Redis,	Server, database, and deployment environment.

Role	Main Responsibilities	System Access
	frontend URL, JWT secret, deployment settings, and API keys.	

Table C.2. Responsibility matrix.

Task	Registered User	Project Creator	Collaborator	AI Assistant	Administrator
Register and login	Yes	Yes	Yes	No	No
Create project	Yes	Yes	No	No	No
Invite collaborator	No	Yes	Limited by project policy	No	No
Send project message	Yes	Yes	Yes	No	No
Generate code using @ai	Yes	Yes	Yes	Yes, as processor	No
Edit and save files	Yes	Yes	Yes	No	No
Configure deployment	No	No	No	No	Yes

Appendix D: Hardware and Software Requirements

The following requirements describe the minimum and recommended environment for running CoBuilder during local development, academic demonstration, and lightweight deployment.

D.1 Hardware Requirements

Table D.1. Hardware requirements.

Component	Minimum Requirement	Recommended Requirement	Purpose
Processor	Dual-core CPU	Quad-core CPU or higher	Runs browser, Node.js server, and development tools smoothly.
RAM	4 GB	8 GB or higher	Supports frontend development server, backend server, and browser tabs.
Storage	1 GB free disk space	5 GB free disk space	Stores source code, dependencies, report files, and generated assets.
Network	Stable internet connection	Broadband internet connection	Required for Gemini API calls, deployment, package installation, and real-time collaboration.
Client Device	Laptop or desktop	Modern laptop or desktop	Provides a usable screen size for code workspace and screenshots.

D.2 Software Requirements

Table D.2. Software requirements.

Software	Version / Type	Usage in Project
Node.js	18 or later	Runs the Express backend and Vite frontend tooling.
MongoDB	Local or cloud database	Stores users, projects, messages, collaborators, and file tree data.
Redis	Optional local or cloud instance	Supports token blacklisting during logout when connected.
React.js	Version 19 used in project	Builds the frontend user interface.
Vite	Version 6 used in project	Provides frontend development and build tooling.
Express.js	Version 5 used in project	Implements REST API routes and middleware.
Socket.IO	Version 4 used in project	Provides real-time chat and file-tree synchronization.
Google Gemini API	API key required	Provides AI-assisted code generation.
Web Browser	Chrome, Edge, or Firefox	Runs the CoBuilder frontend application.

D.3 Environment Variables

Table D.3. Important environment variables.

Variable	Used By	Purpose
PORT	Backend	Defines backend server port, commonly 3001.
MONGO_URI	Backend	MongoDB connection string.
JWT_SECRET	Backend	Secret key used to sign and verify JWT tokens.
GOOGLE_API_KEY	Backend	API key for Gemini AI model access.
FRONTEND_URLS	Backend	Allowed CORS origins for frontend clients.
VITE_API_URL	Frontend	Base URL used by Axios and Socket.IO client.

Appendix E: System Screenshots

This appendix is reserved for screenshots captured from the final working CoBuilder system. The screenshots should be pasted below the corresponding captions. Each figure should be clear, cropped properly, and large enough to show the relevant user interface elements.

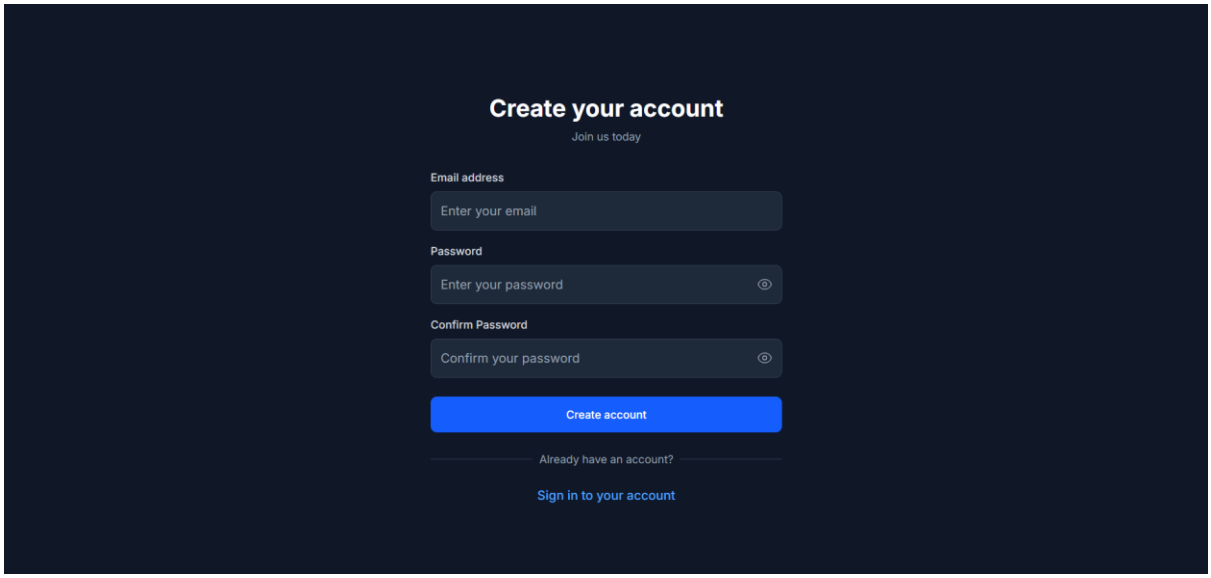

Fig. E.1. User registration screen.

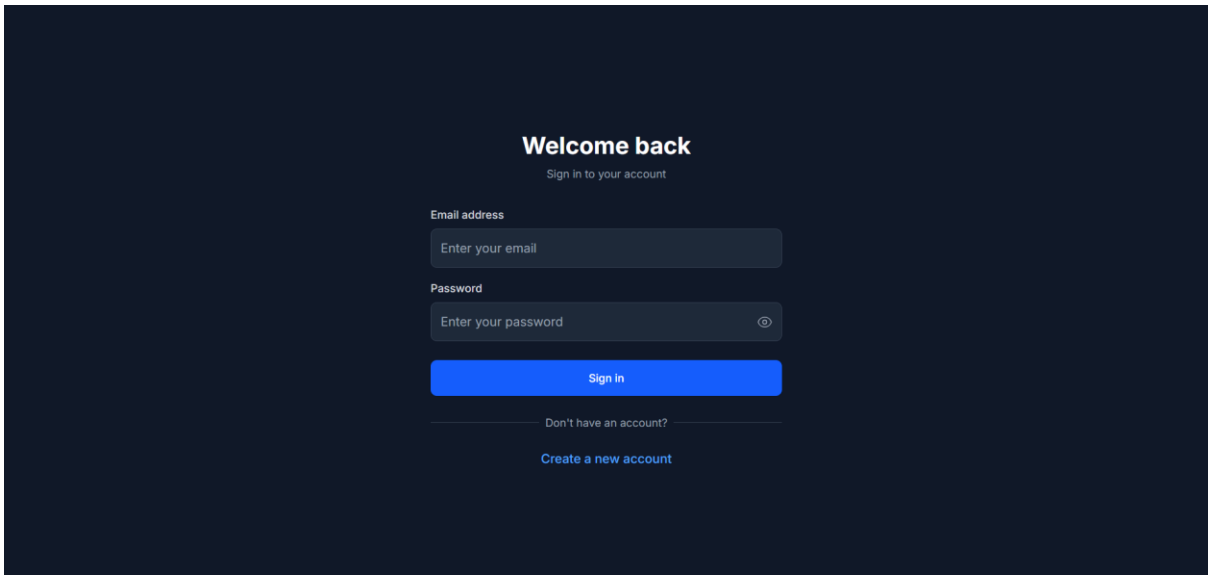

Fig. E.2. User login screen.

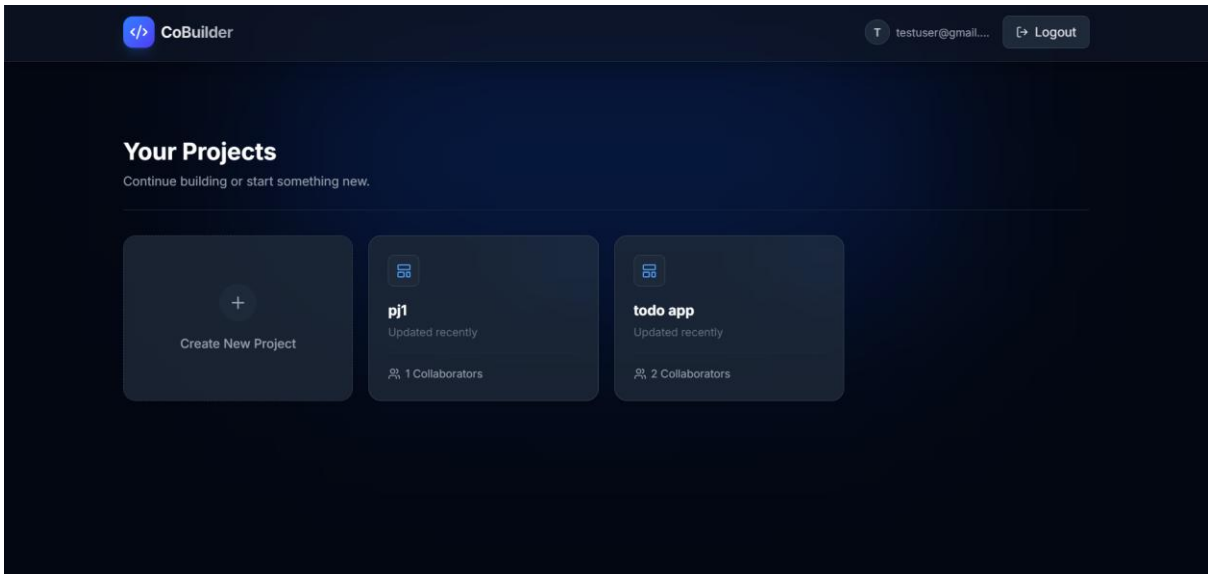

Fig. E.3. Project dashboard showing created projects.

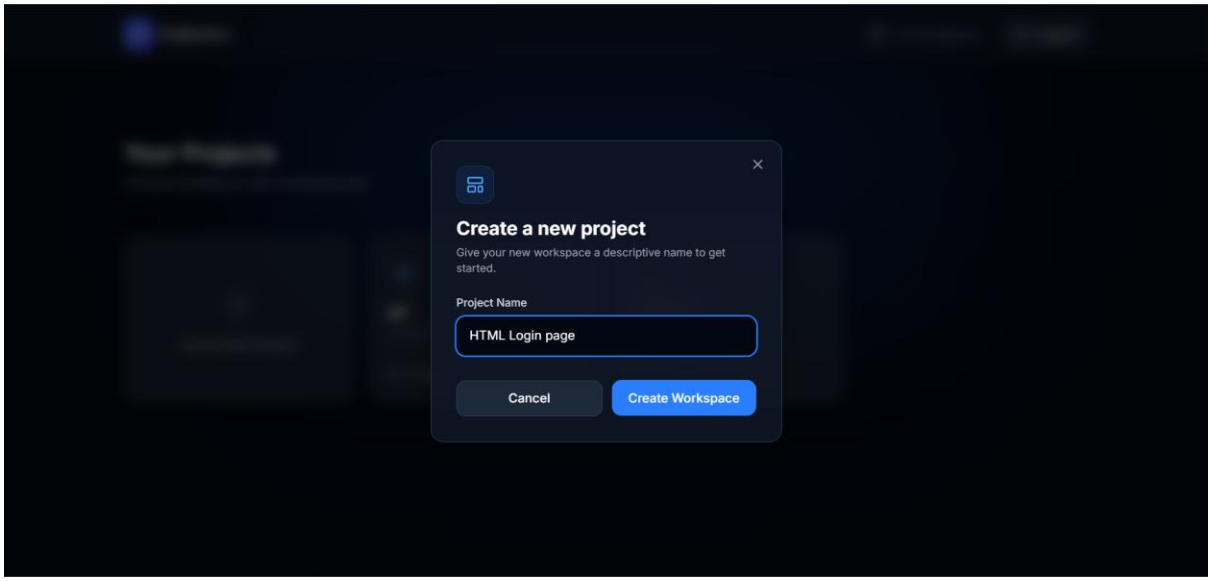

Fig. E.4. Create project dialog.

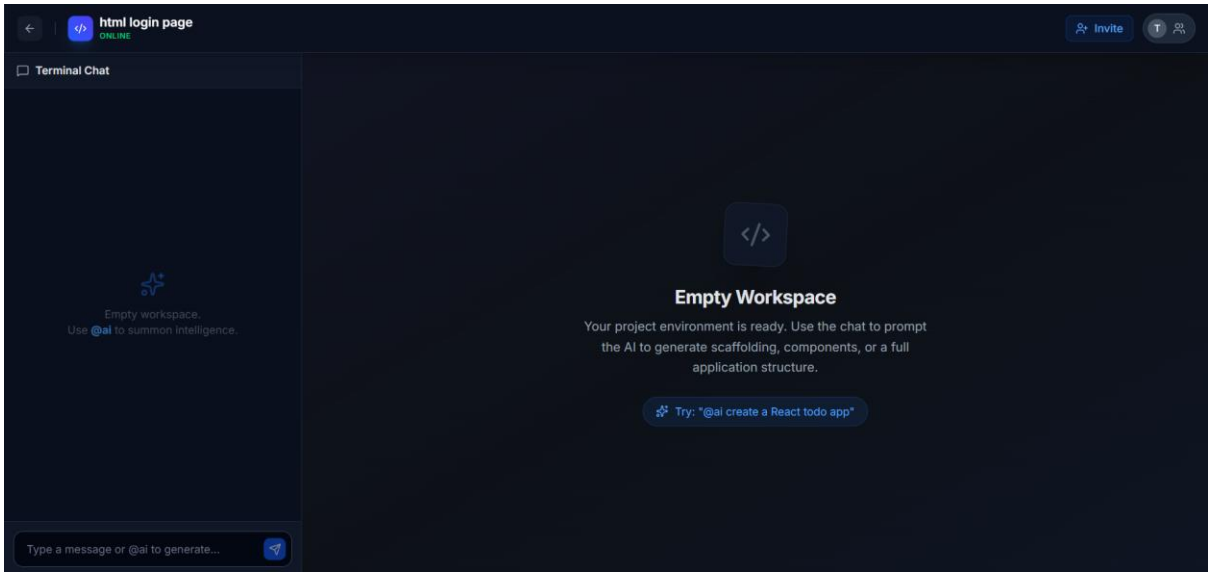

Fig. E.5. Project workspace with chat panel.

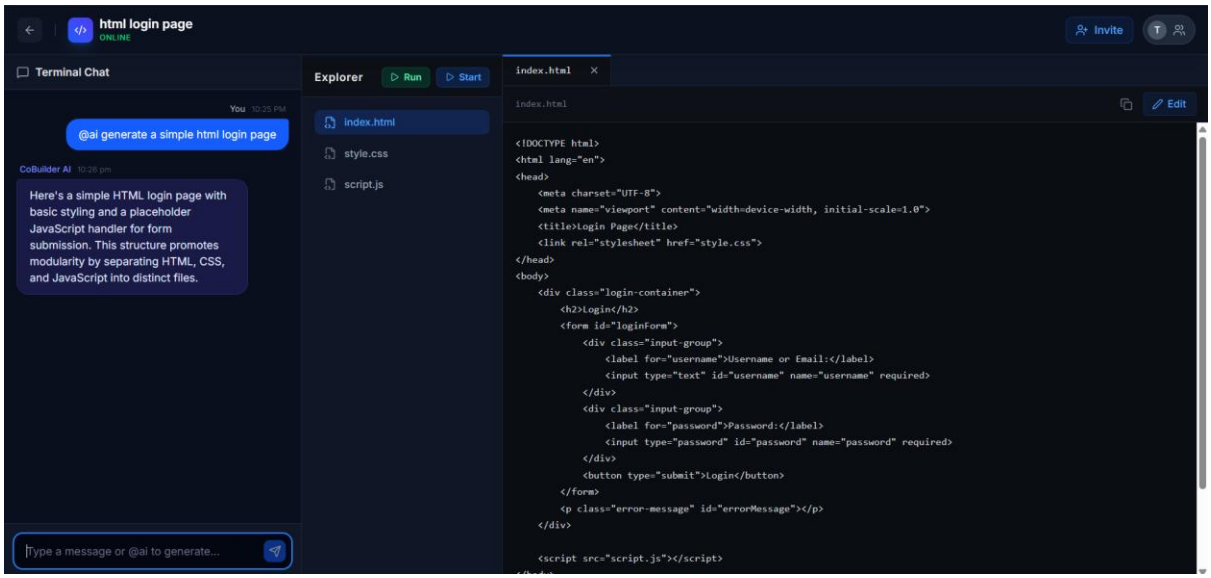

Fig. E.6. AI generated file tree and code viewer.

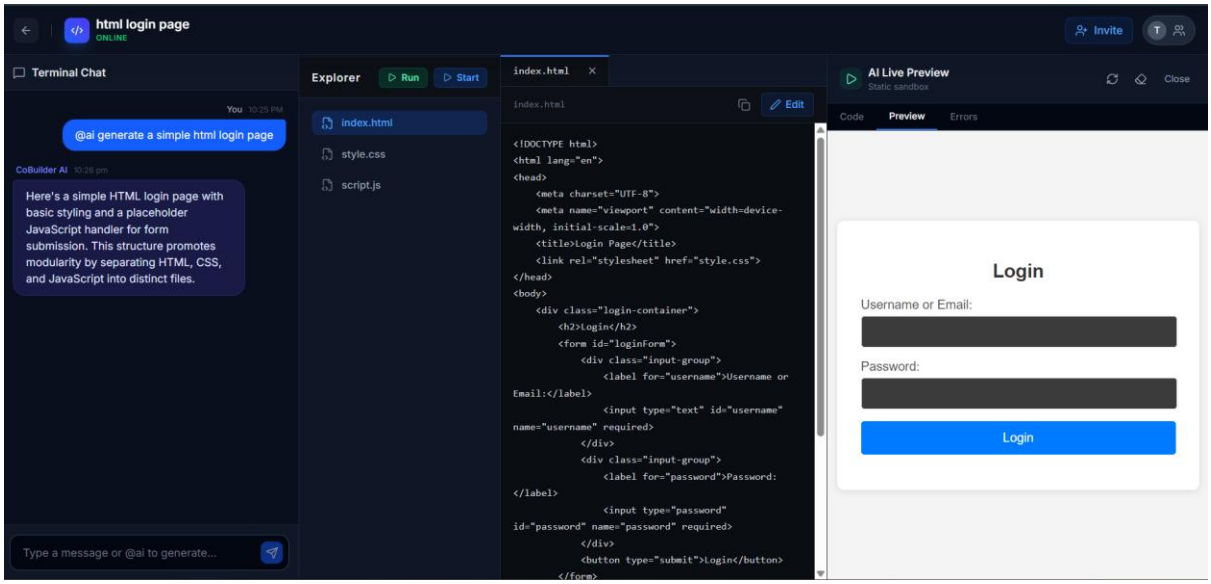

Fig. E.7.Code running in browser

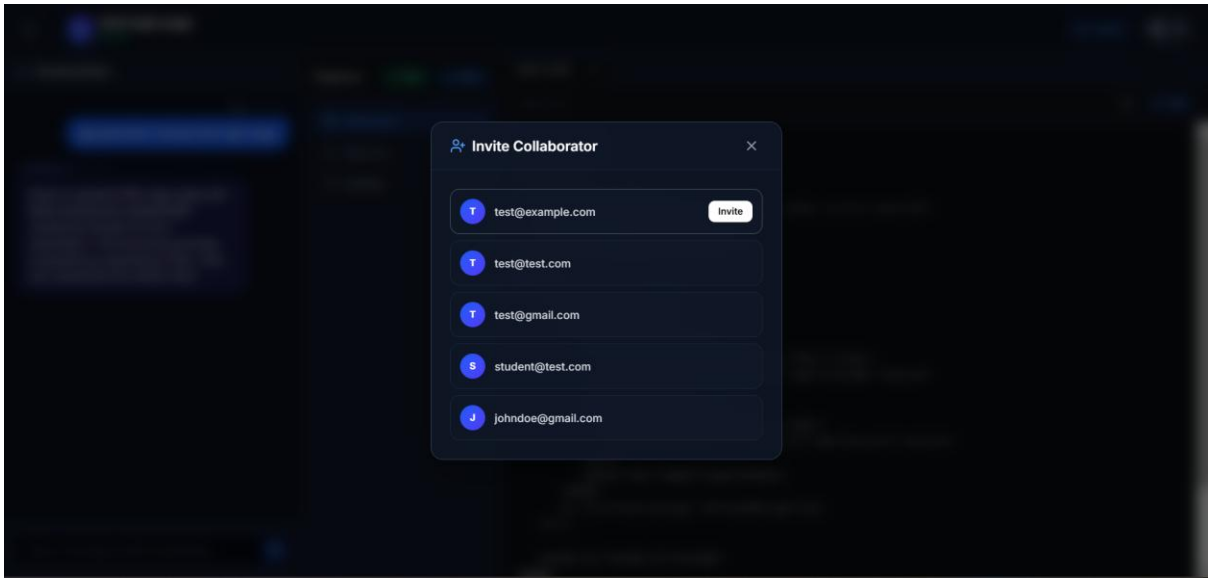

Fig. E.8. Invite collaborator dialog.